

Artificial Neural Networks

Nguyễn Thị Hải Bình

nth.binh@hutech.edu.vn

Google classroom: [hnyvemh](#)

Contents

- Neural Network - Model Representation
- Backpropagation - Method for Training a Neural Network
- Hands-on practices

Neural Network

- Origins: Algorithms that try to mimic the brain.
- Was very widely used in 80s and early 90s; popularity diminished in late 90s.
- Recent resurgence: State-of-the-art technique for many applications

Model Representation

Neuron in The Brain

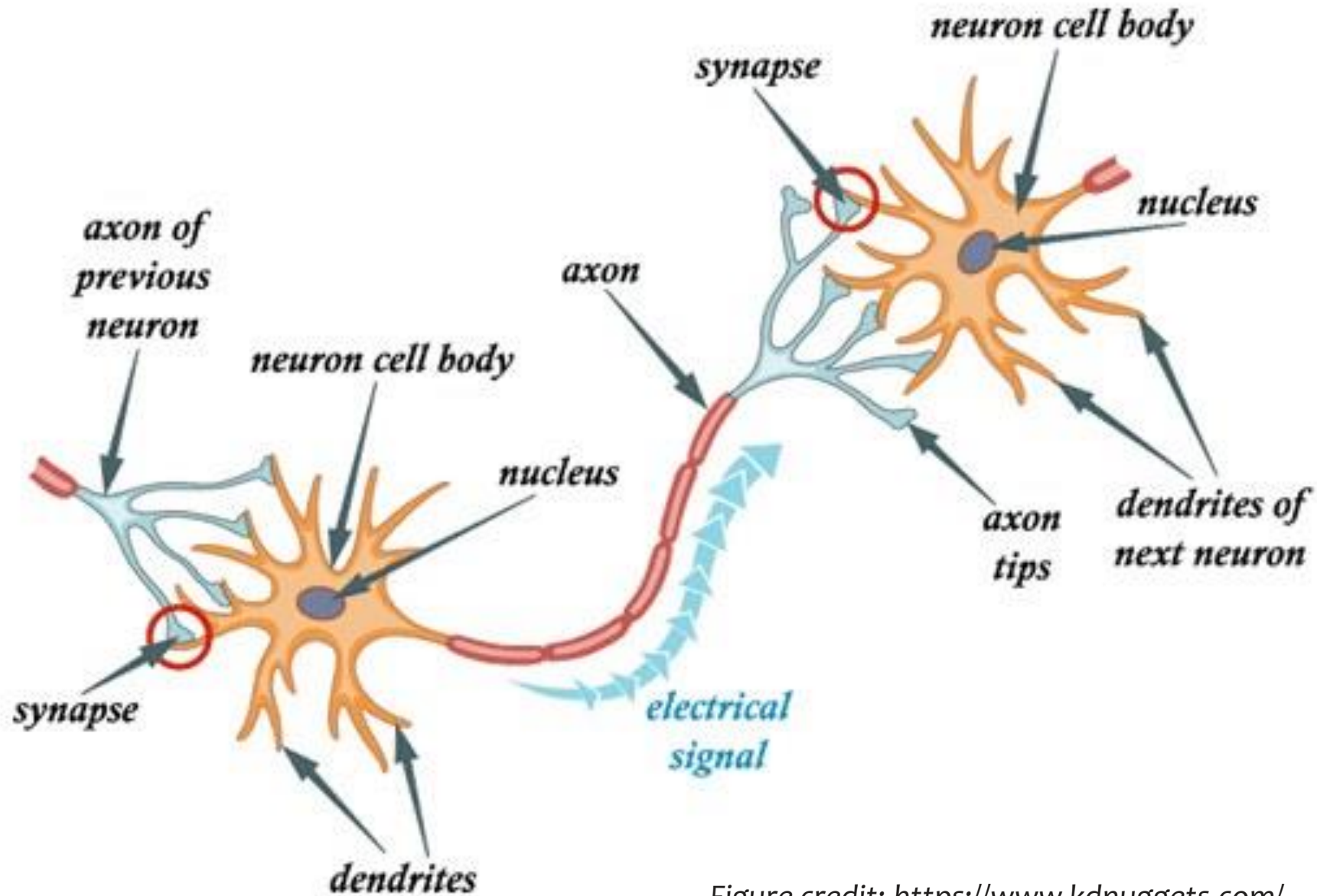
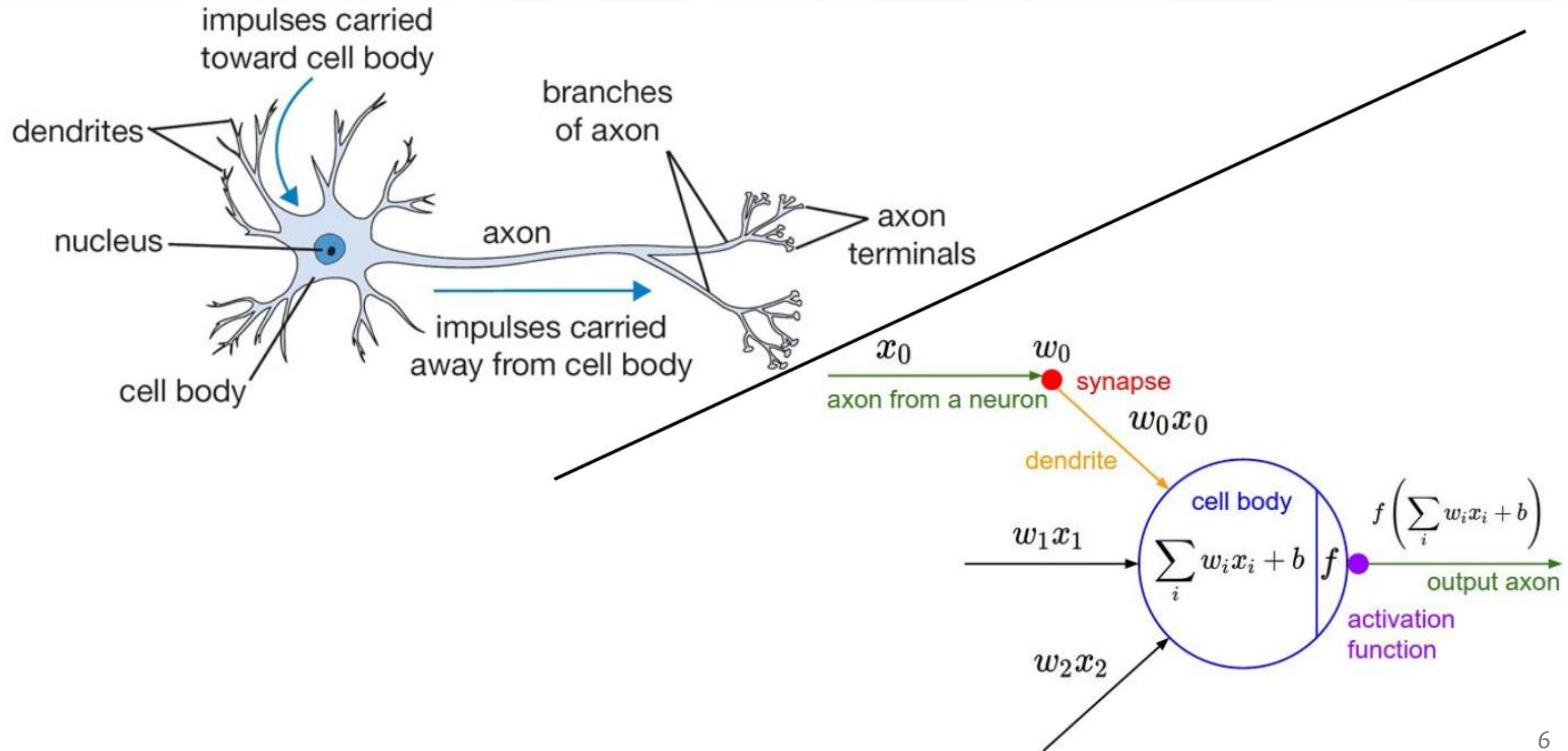


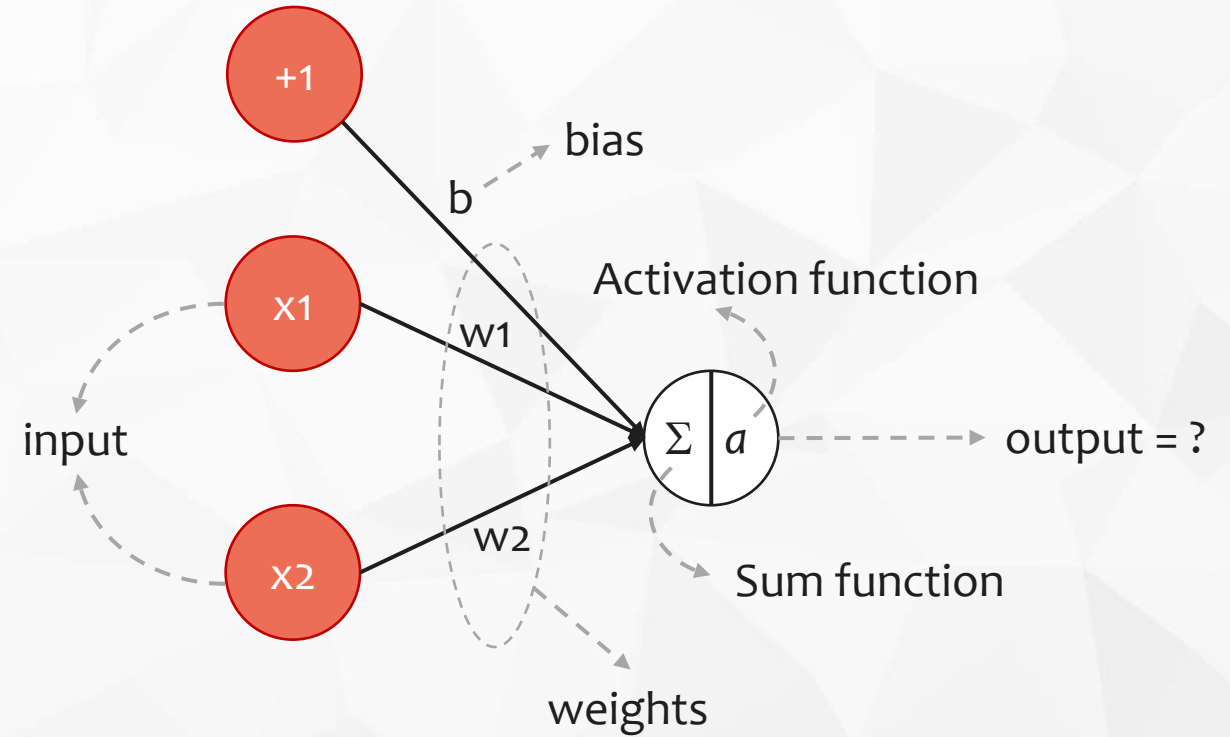
Figure credit: <https://www.kdnuggets.com/>

Artificial Neuron



Component of ANN

- Neurons
 - Input neurons
 - Hidden neurons
 - Output neurons
- Activation function:
 - Used to generate output from the previous neuron.
- Weights
- Biases



$$output = a(x_1 \times w_1 + x_2 \times w_2 + b)$$

```
# Effect of bias adjustments
```

```
x = np.arange(-8, 8, 0.1)
```

```
w1 = 5
```

```
b = -8
```

```
s = x*w1 + b
```

```
y = sigmoid(s)
```

```
plt.plot(x, y, label="w1 = 5, b = -8")
```

```
w1 = 5
```

```
b = 0
```

```
s = x*w1 + b
```

```
y = sigmoid(s)
```

```
plt.plot(x, y, label="w1 = 5, b = 0")
```

```
w1 = 5
```

```
b = 8
```

```
s = x*w1 + b
```

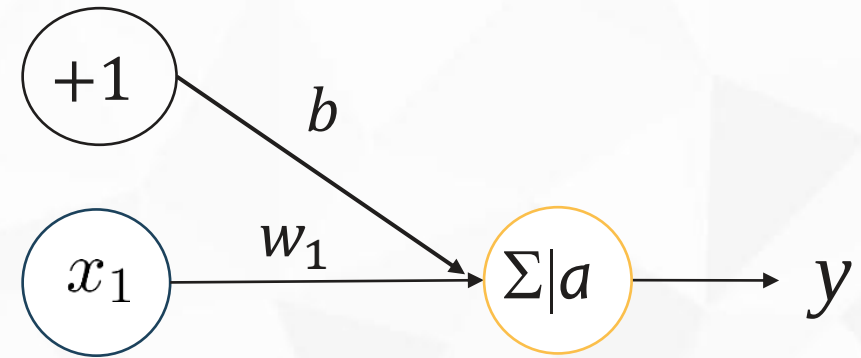
```
y = sigmoid(s)
```

```
plt.plot(x, y, label="w1 = 5, b = 8")
```

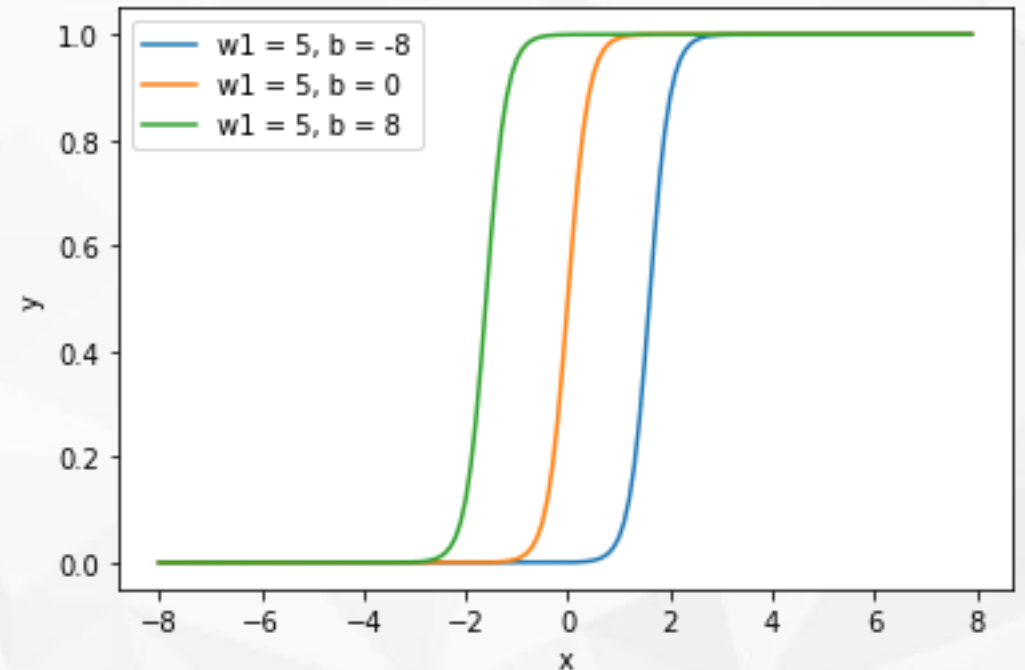
```
plt.xlabel('x')
```

```
plt.ylabel('y')
```

```
plt.legend(loc=2)
```

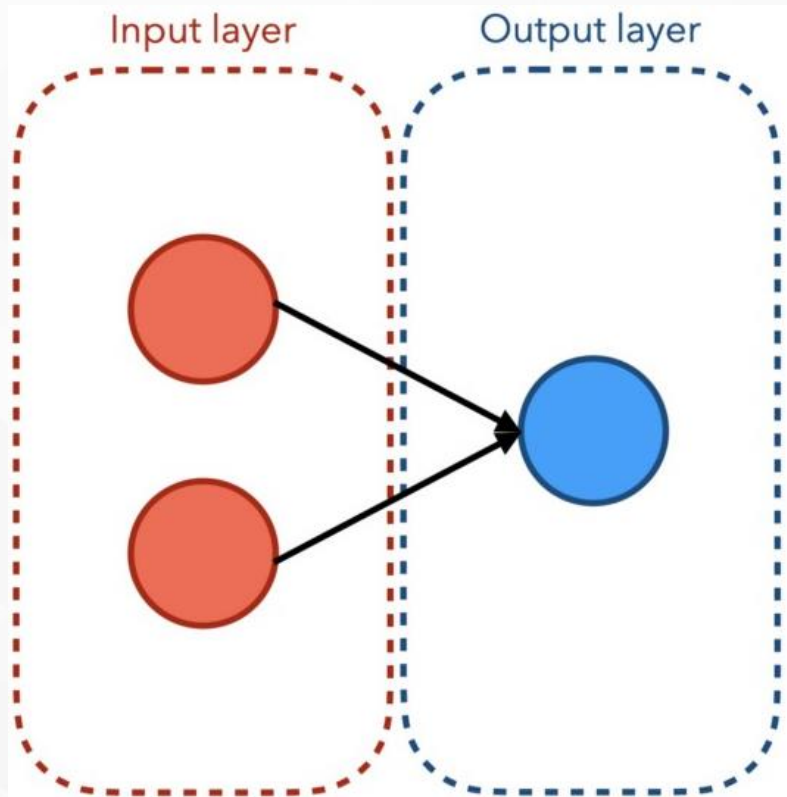


$$y = \text{sigmoid}(x_1 \times w_1 + b)$$

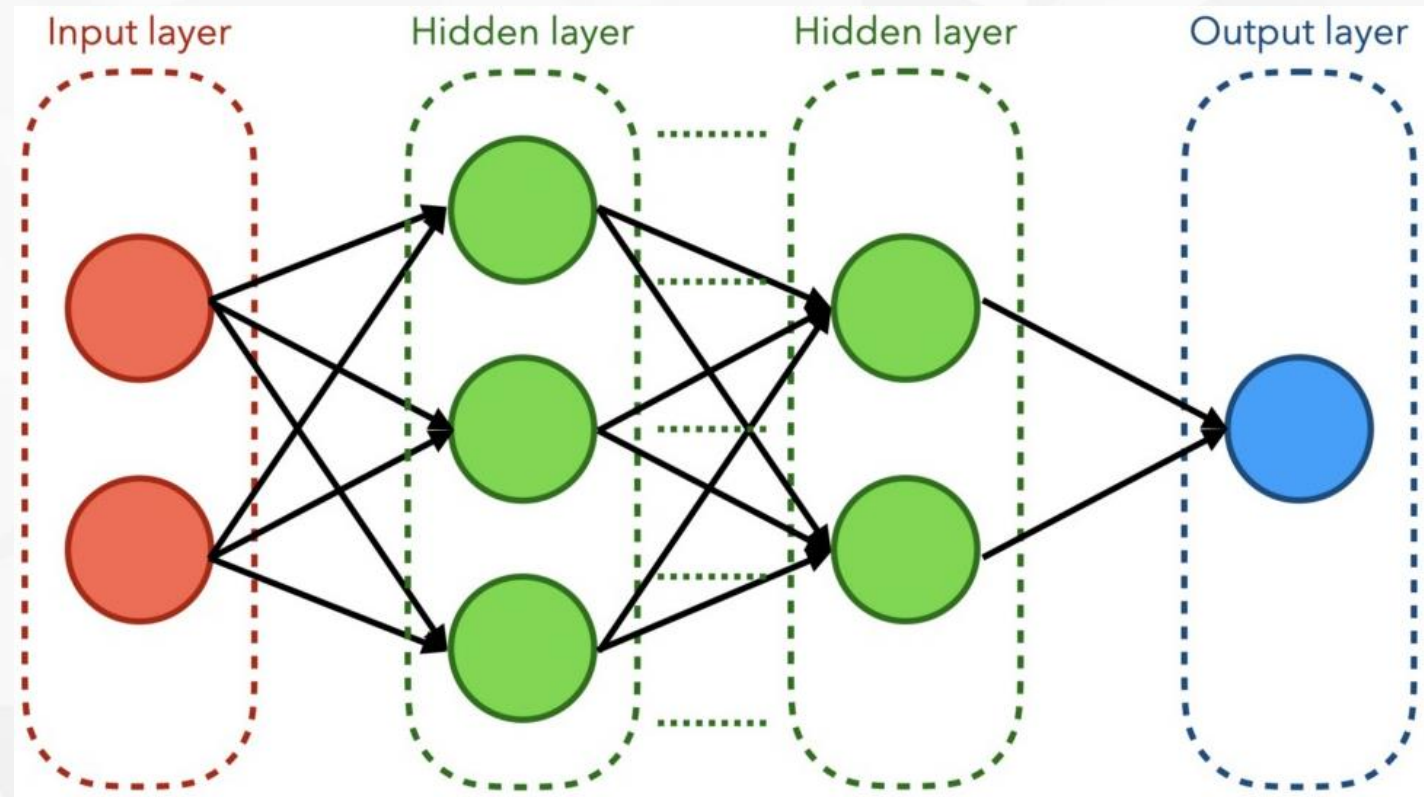


Layer

- A layer consists of one or more neurons.
- There are three layers in ANN: Input, hidden, and output layer.



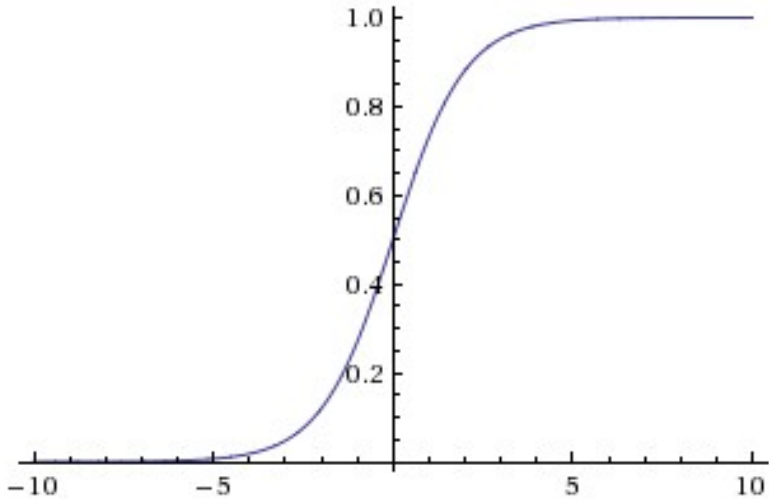
Single-layer perceptron



Multi-layer perceptron

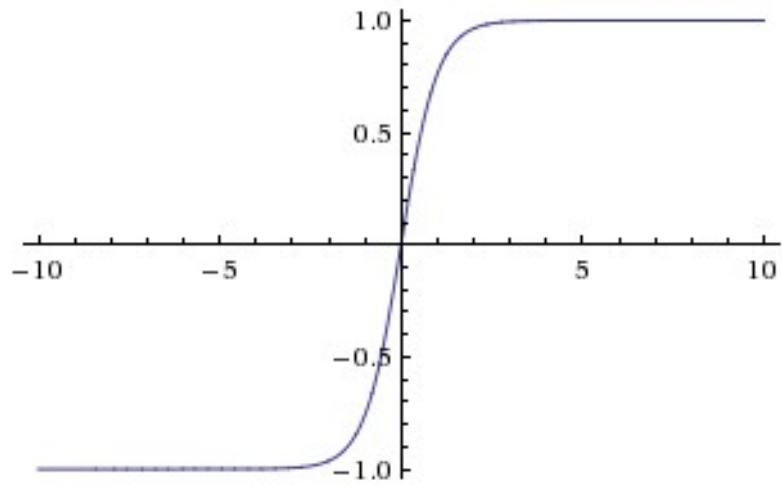
Activation Functions

- Historically people use **Sigmoid** and **tanh**
- From 2012 **ReLU** became quite popular (default recommendation)
- A few kind of hipster activation functions: **Leaky Relu**, **Maxout**, and **ELU**



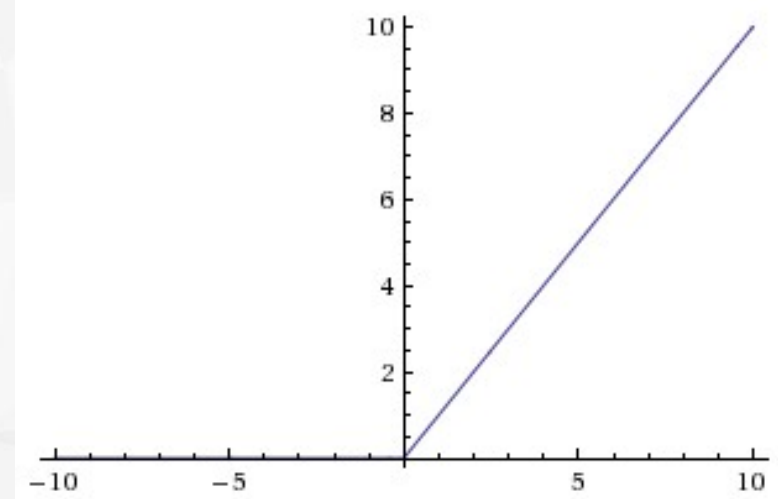
Sigmoid function

$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-\theta^T x}}$$



Tanh function

$$f(x) = \tanh(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

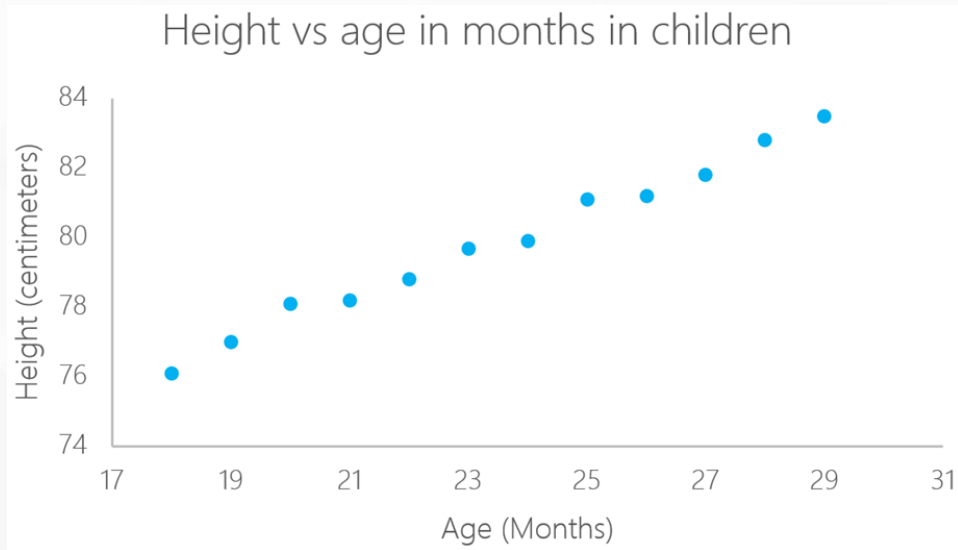


ReLU function

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

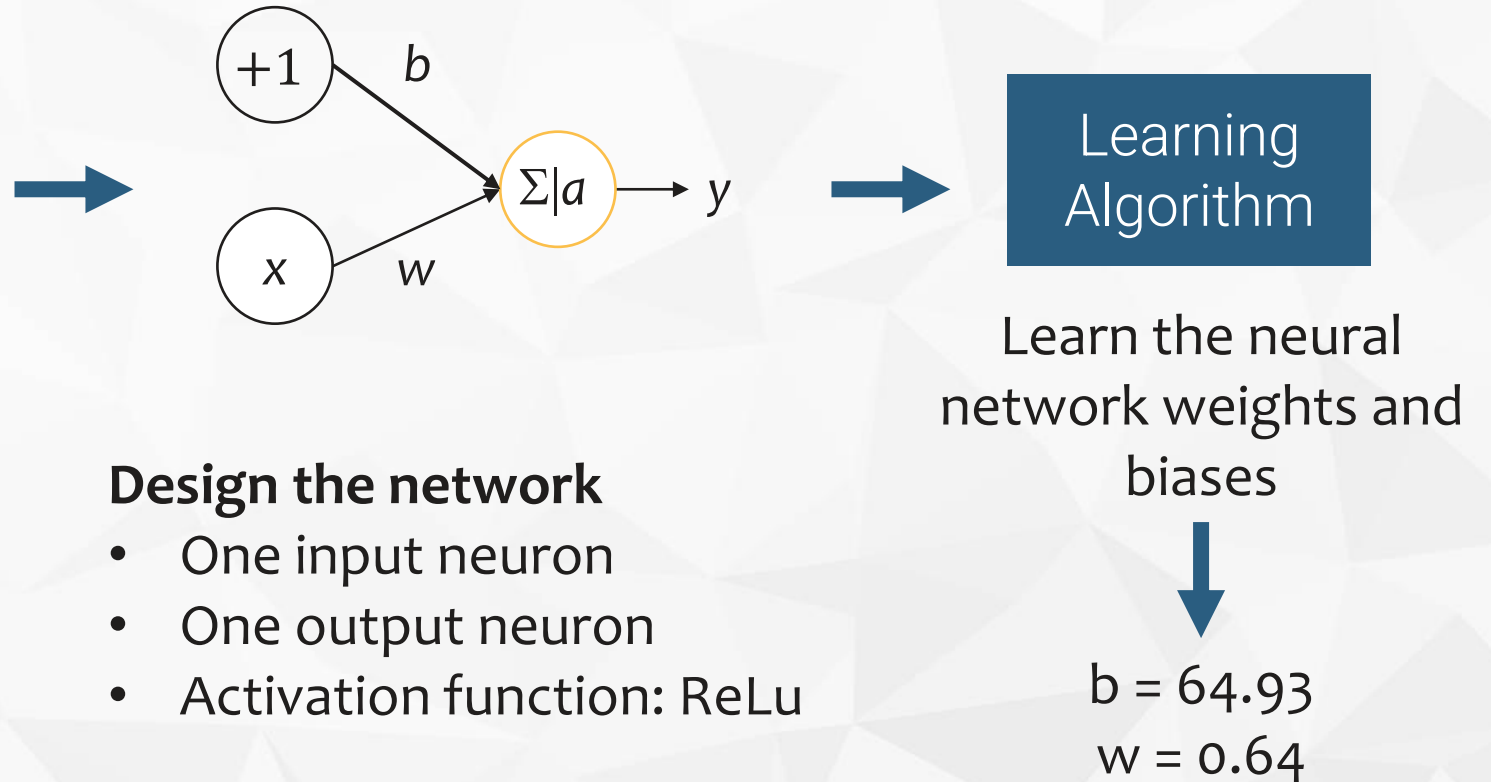
Neural Network Example: predict baby heights

Training phase: to find the best neural network for estimating baby heights given baby ages.



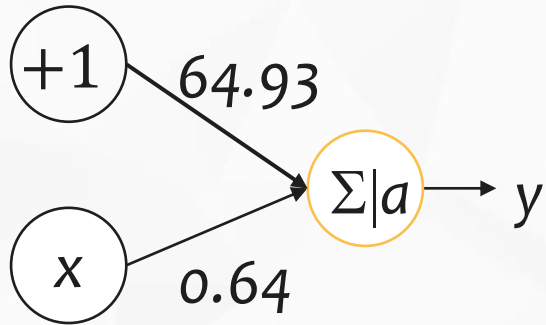
Training data: (x^i, y^i) ($n = 12$)

- x^i – age of the i^{th} baby,
- y^i – height of the i^{th} baby



Neural Network Example: predict baby heights

Prediction phase: Predict baby heights given baby ages using the trained network.



The trained model

Given: baby age = 48 months

→ Predicted baby height = ?

$$\begin{aligned}\text{Predicted baby height} &= \text{ReLu}(0.64 * x + 64.93) \\ &= \text{ReLu}(95.65) = 95.65\end{aligned}$$

Neural Network Example: predict iris flower type

Samples

(instances, observations)

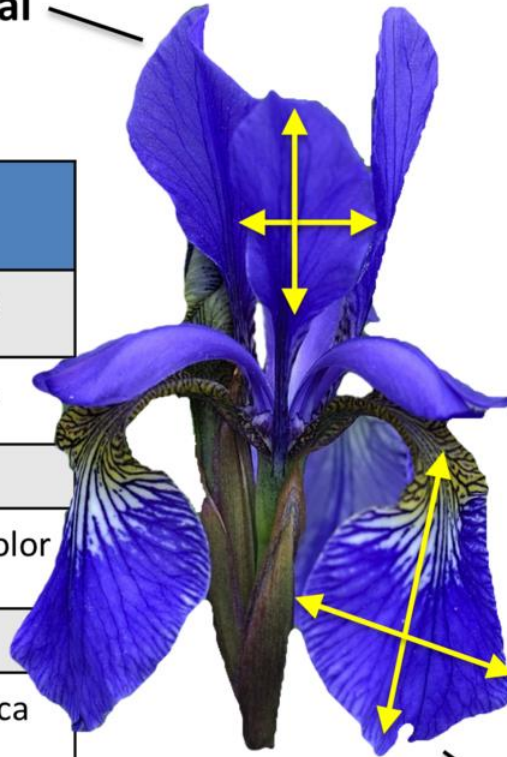
	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

Features: sepal length, sepal width, petal length, and petal width

Class labels: Setosa, Versicolour, and Virginica

Petal

Sepal



One hot encoding:

Setosa = $[1, 0, 0]$

Versicolour = $[0, 1, 0]$

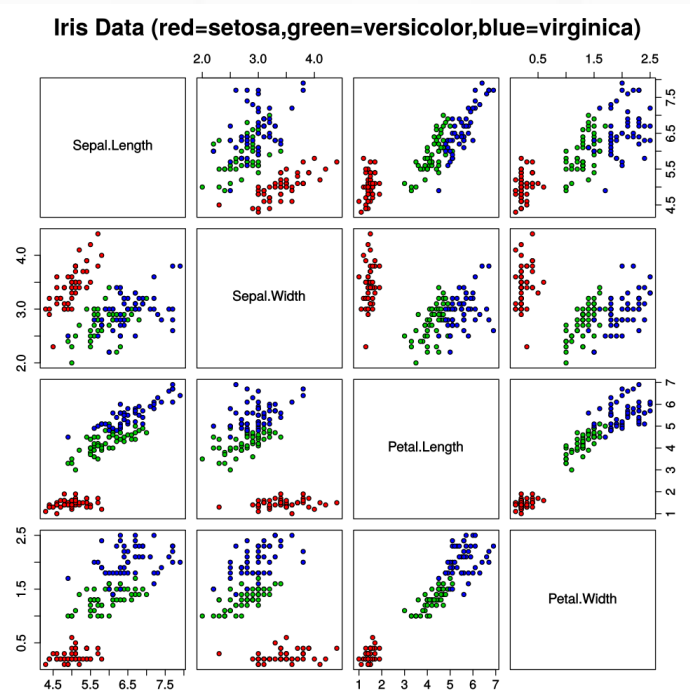
Virginica = $[0, 0, 1]$

Training data: (x^i, y^i) ($n = 150$)

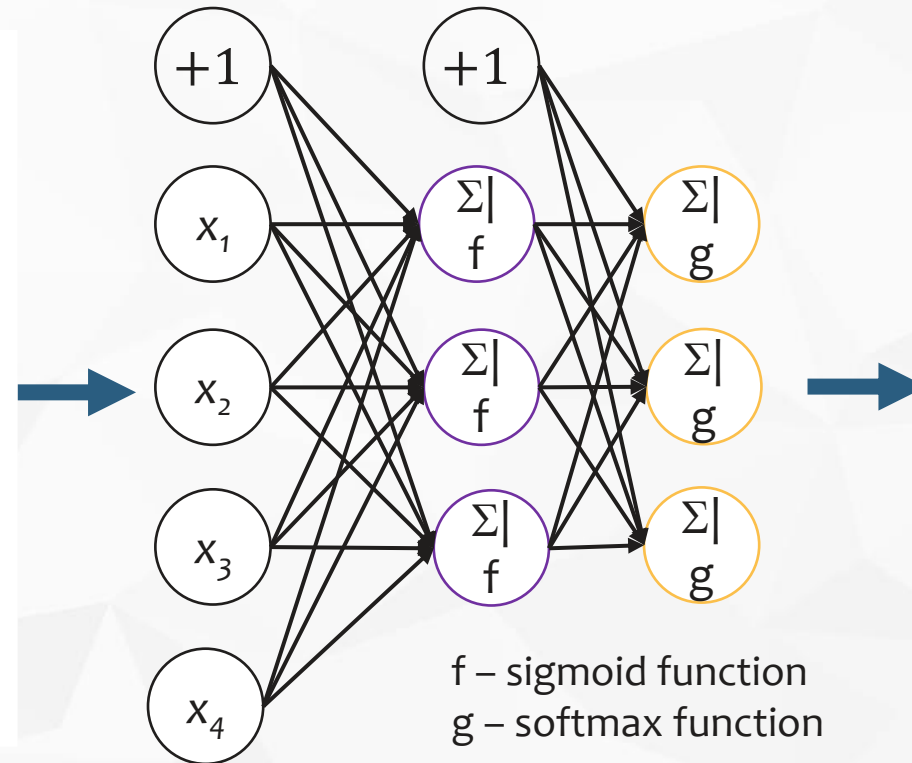
- $x^i = [x^i_1, x^i_2, x^i_3, x^i_4]$
- $y^i = [y^i_1, y^i_2, y^i_3]$

Neural Network Example: predict iris flower type

Training phase: to find the best neural network for predicting iris flower types given petal and sepal lengths.



Training data: (x^i, y^i) ($n = 150$)



f – sigmoid function
 g – softmax function

$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix} \xrightarrow{z} \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \rightarrow \begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix} g(z)$$

Learning Algorithm

Learn the neural network weights and biases

Weights between input and hidden layer:

$\begin{bmatrix} 1.15 & -0.29 & 0.01 \\ 1.44 & 1.53 & -1.41 \\ -1.9 & -1.27 & 0.68 \\ -2.21 & -1.35 & 1.58 \end{bmatrix}$

Biases between input and hidden layer:

$\begin{bmatrix} 1.46 & 1.19 & -0.94 \end{bmatrix}$

Weights between hidden and output layer:

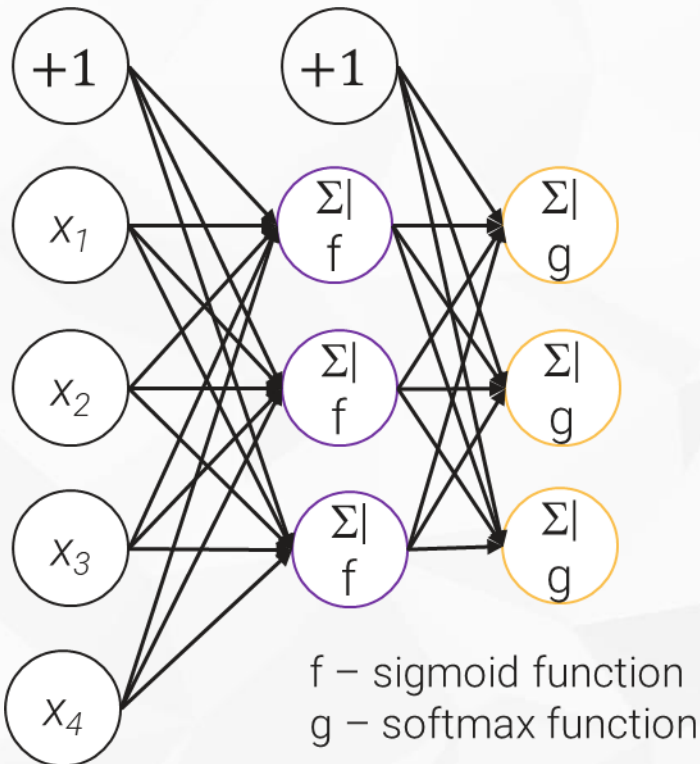
$\begin{bmatrix} 1.04 & 1.06 & -2.01 \\ 2.45 & -2.1 & -1.65 \\ -1.94 & 0.36 & 2.06 \end{bmatrix}$

Biases between hidden and output layer:

$\begin{bmatrix} -0.81 & 0.55 & 0.94 \end{bmatrix}$

Neural Network Example: predict iris flower type

Prediction phase: predict iris flower types given petal and sepal lengths using the trained network.



$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix} \xrightarrow{z} \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \xrightarrow{g(z)} \begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

Weights between input and hidden layer **w0**:

```
[ 1.15 -0.29  0.01]
[ 1.44  1.53 -1.41]
[-1.9   -1.27  0.68]
[-2.21 -1.35  1.58]
```

Biases between input and hidden layer **b0**:

```
[ 1.46  1.19 -0.94]
```

Weights between hidden and output layer **w1**:

```
[ 1.04  1.06 -2.01]
[ 2.45 -2.1  -1.65]
[-1.94  0.36  2.06]
```

Biases between hidden and output layer **b1**:

```
[-0.81  0.55  0.94]
```

Given: $x = [6.3, 2.3, 4.4, 1.3]$

→ Predicted flower type = ?

$h = \text{sigmoid}(x @ w_0 + b_0)$
 $= [0.69, 0.01, 0.72]$

$o = \text{softmax}(h @ w_1 + b_1)$
 $= [0.03, 0.6, 0.37]$

→ Predicted flower type = $[0, 1, 0]$

Lab: Neural Networks - Forward Propagation

- In this lab, we will build a neural network from scratch and code how it performs predictions using forward propagation.
- Please note that all deep learning libraries have the entire training and prediction processes implemented, and so in practice you wouldn't really need to build a neural network from scratch.
- Hopefully completing this lab will help you understand neural networks and how they work even better.


```

1 # import required packages
2 import numpy
3 # define mlp class
4 class mlp():
5     # define the network with the given weights and biases
6     # input size = 4, output size = 3
7     # number of hidden layers = 1, number of neurons in the hidden layer = 3
8     # activation function in the hidden layer = sigmoid function
9     # activation function in the output layer = softmax function
10    def __init__(self):
11        # weights
12        w0 = [[ 1.15, -0.29,  0.01], [ 1.44,  1.53, -1.41], [-1.9,  -1.27,  0.68], [-2.21, -1.35,  1.58]]
13        w1 = [[ 1.04,  1.06, -2.01], [ 2.45, -2.1,  -1.65], [-1.94,  0.36,  2.06]]
14        w = [w0, w1]
15        # biases
16        b = ...
17        self.weights = w
18        self.biases = b
19        # sigmoid function
20        def sigmoid(self, z):
21            ...
22        # softmax function
23        def softmax(self, x):
24            ...
25        # feedforward
26        # return values of the hidden and output layer
27        def feedforward(self, input):
28            ...
29        # predict flower type
30        # return class name (class name = {'sentosa', 'versicolor', 'virginica'})
31        def predict(self, input):
32            ...

```

Backpropagation - Method for Training a Neural Network

Goal of backpropagation:

To optimize the weights so that the neural network can learn how to correctly map arbitrary inputs to outputs.

Gradient Descent: simple explanation

- Given a training set consisting of m observations: (x^i, y^i) ($1 \leq i \leq m$)
- Assume that y^i can be predicted using the following network:



f - linear function
output of the network $= f(x*w) = x*w$

- Find w ?

→ The idea: Choose w so that $net(x^i)$ is close to y^i , where $net(x^i)$ is the output of the network w.r.t x^i

→ Find w which minimizes the error:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (net(x^i) - y^i)^2$$

↙
 J is call “Loss Function”

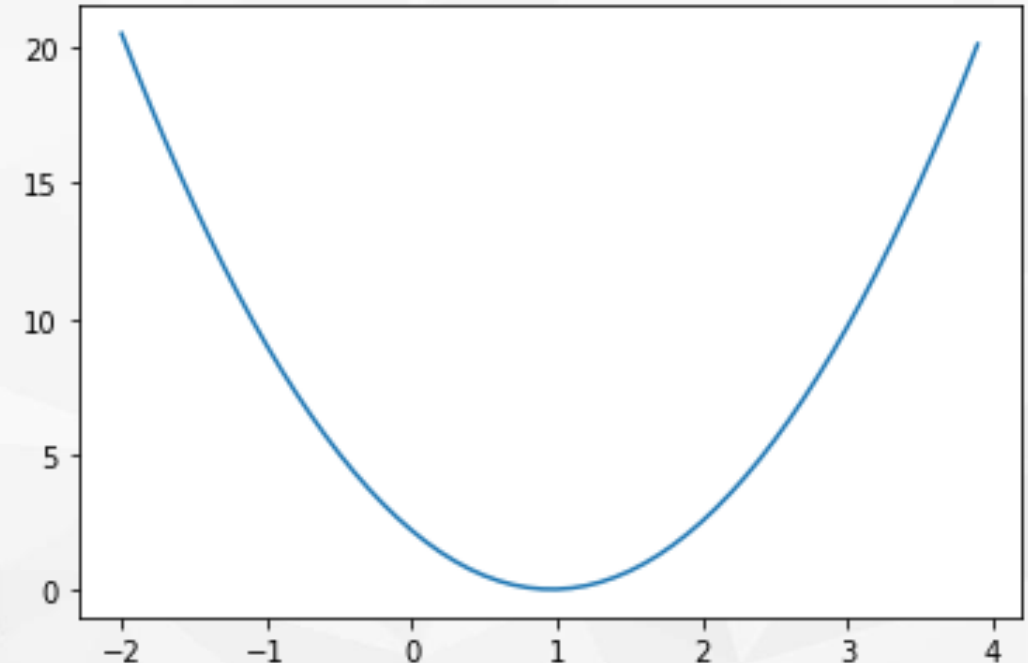
Gradient Descent: simple explanation

- Find w which minimizes the loss function $J(w)$:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (\text{net}(x_i) - y^i)^2$$

- Assume that we have 3 training samples: $(1, 1)$, $(2, 2)$, $(3, 3)$

$$\rightarrow J(w) = 2.33w^2 - 4.66w + 2.33$$



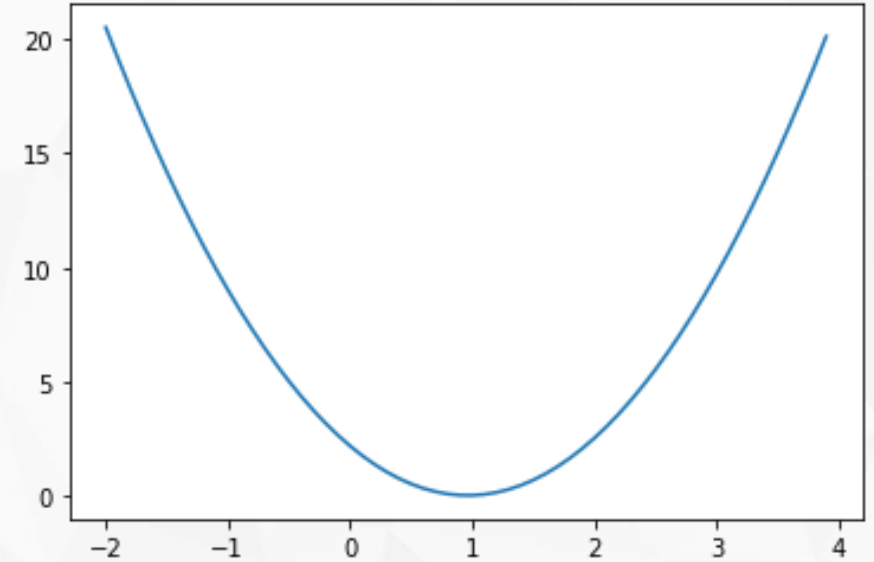
Gradient Descent: simple explanation

- Find w which minimizes the loss function $J(w)$:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (\text{net}(x_i) - y^i)^2$$

- Assume that we have 3 training samples: $(1, 1)$, $(2, 2)$, $(3, 3)$

$$\rightarrow J(w) = 2.33w^2 - 4.5w + 2.19$$



x	$-\infty$	0.96	$+\infty$
$f'(x)$	-	0	+
$f(x)$	$+\infty$	0.019	$+\infty$

Gradient Descent: simple explanation

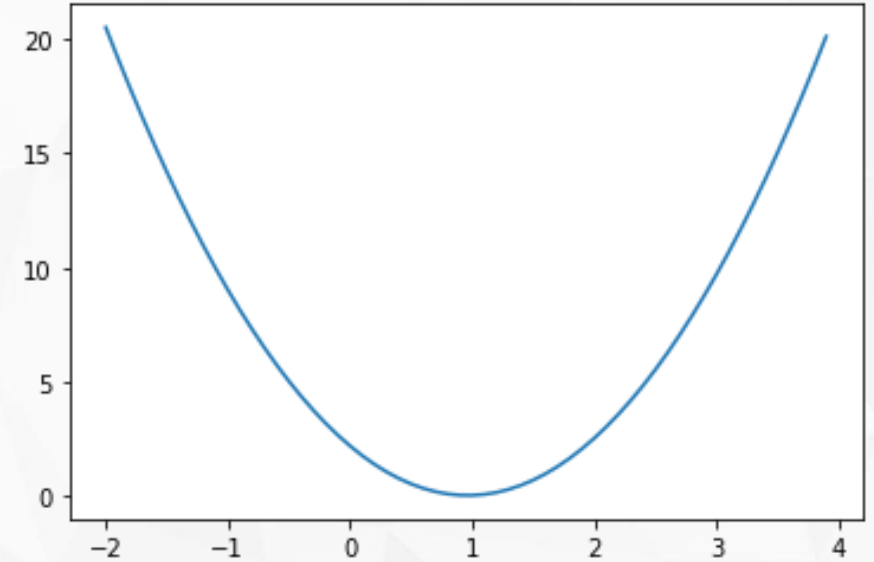
- Find w which minimizes the loss function $J(w)$:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (\text{net}(x_i) - y^i)^2$$

- w is updated using Gradient Descent algorithm:

$$w = w - \alpha \frac{\partial}{\partial w} J(w)$$

α is learning rate ($\alpha > 0$)

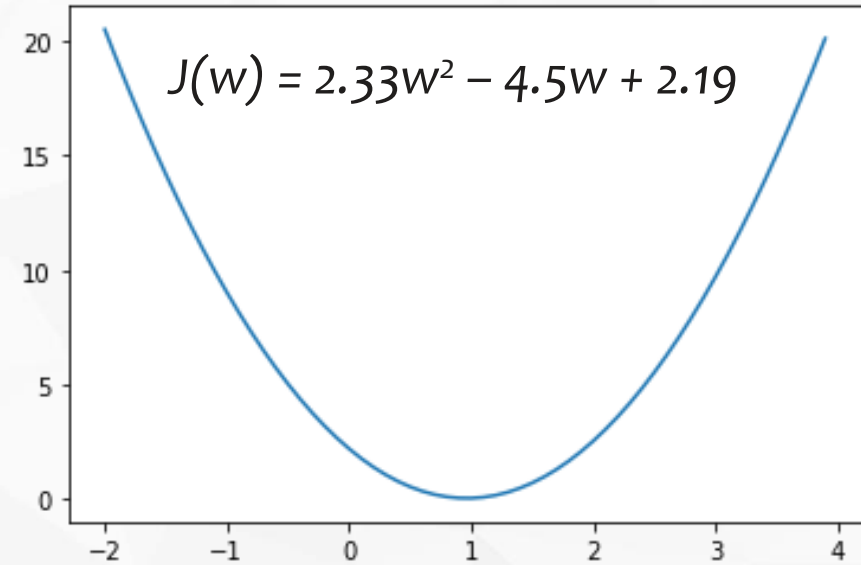


x	$-\infty$	0.96	$+\infty$
$f'(x)$	-	0	+

Gradient Descent: Algorithm

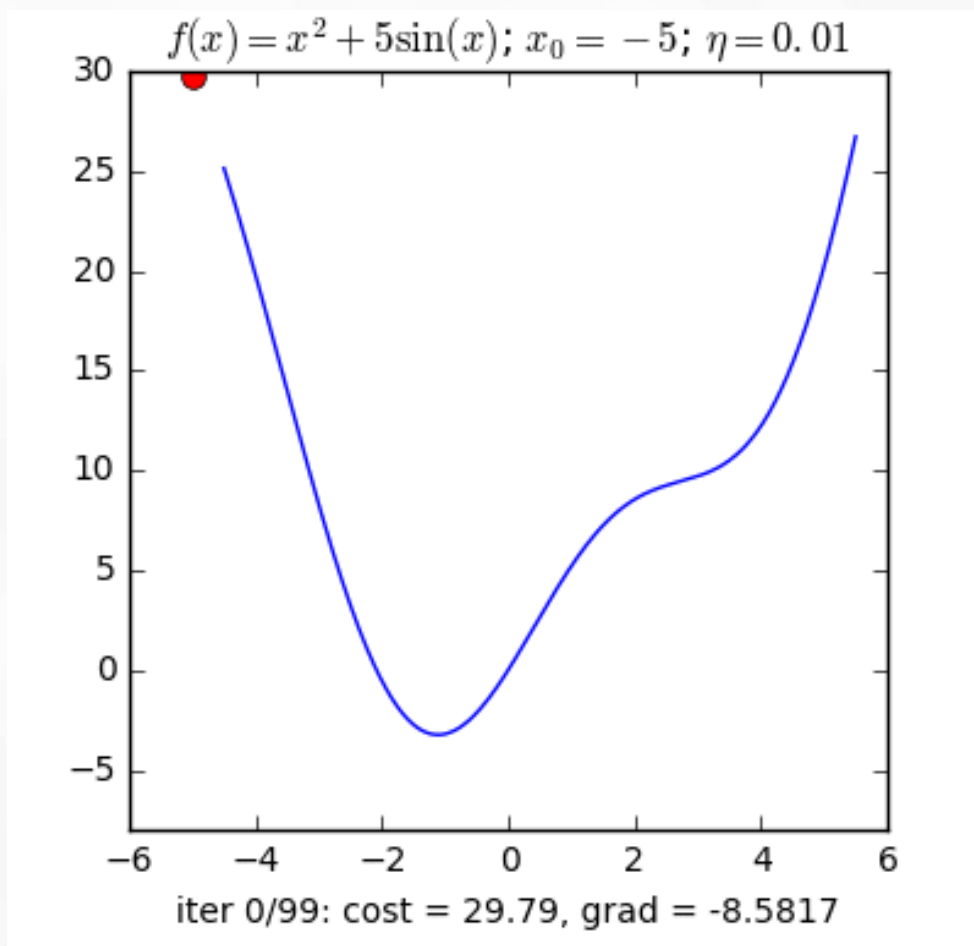
- Initialize the weight values
- Choose α
- Repeat until convergence:

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w)$$

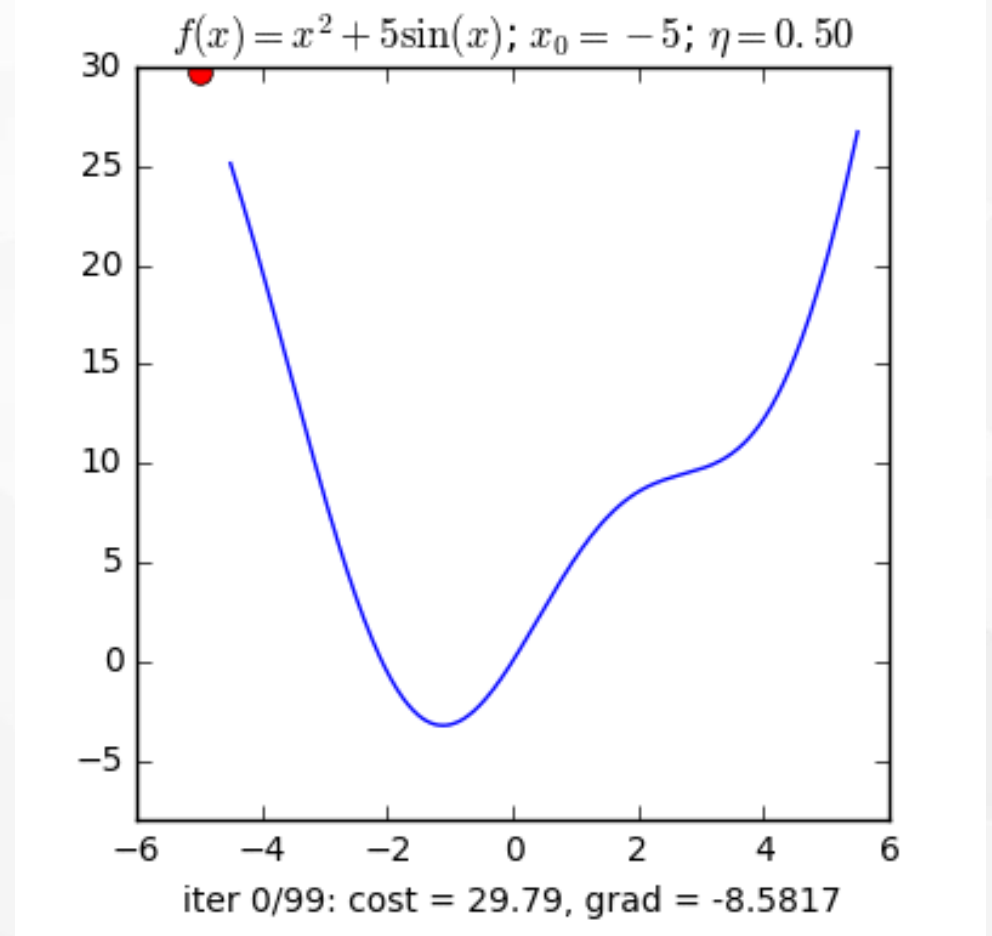


```
def gradientDescent(init_weight, alpha, n_iter):  
    w = init_weight  
    for i in range(n_iter):  
        dw = 2*a*w+b  
        w += -alpha*dw  
    return w
```

Gradient Descent: Learning Rate



If the learning rate is too small, gradient descent can be slow.



If the learning rate is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

Why Backpropagation?

- Neural network loss function:

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

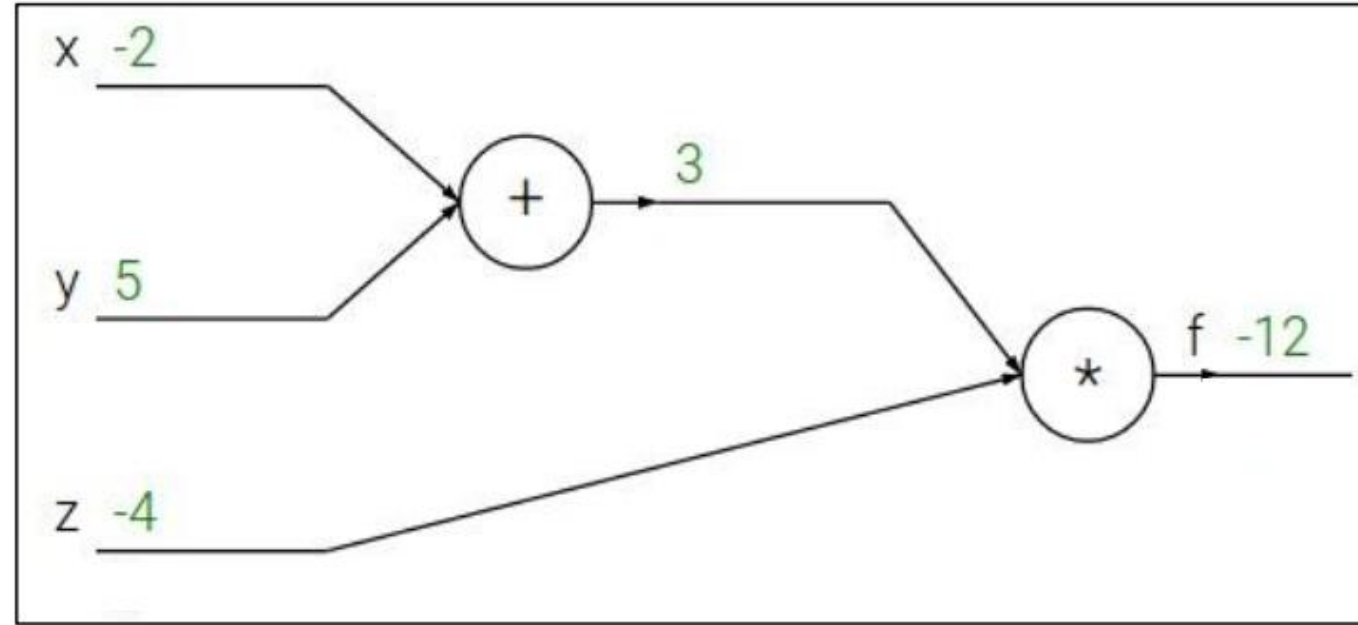
$$h_{\Theta}(x) \in \mathbb{R}^K$$

$$(h_{\Theta}(x))_i = i^{th} \text{ output}$$

Backpropagation: simple example

$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$



Backpropagation: simple example

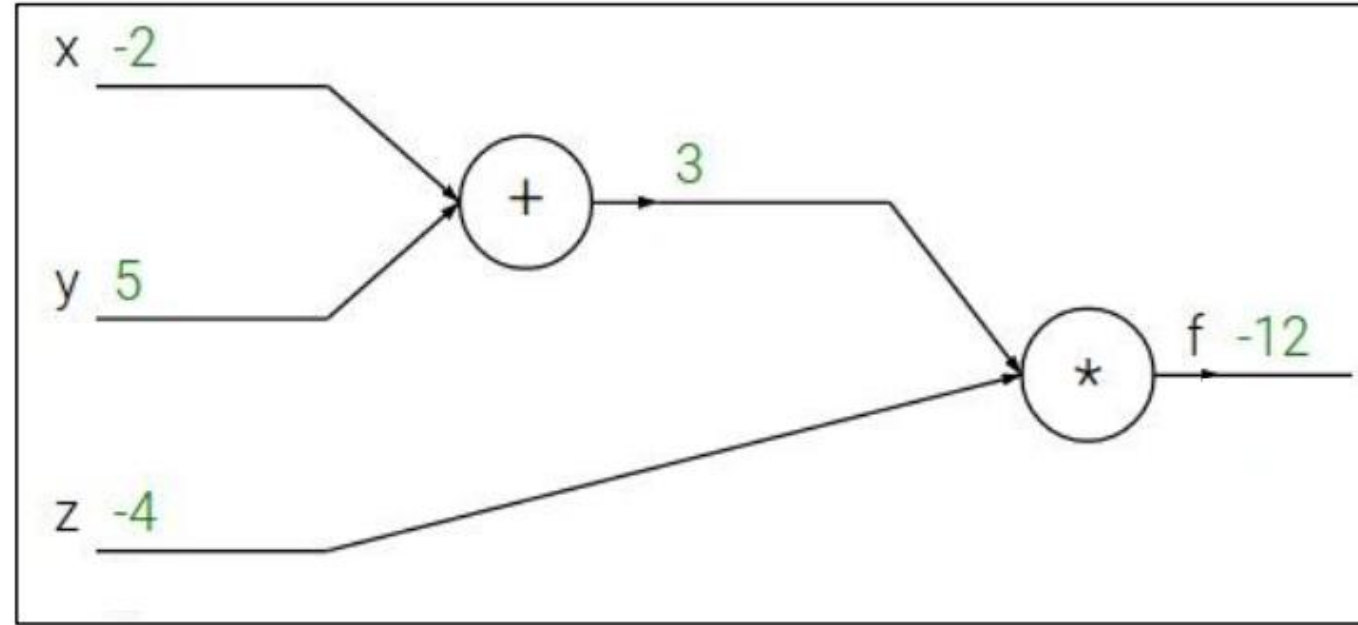
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: simple example

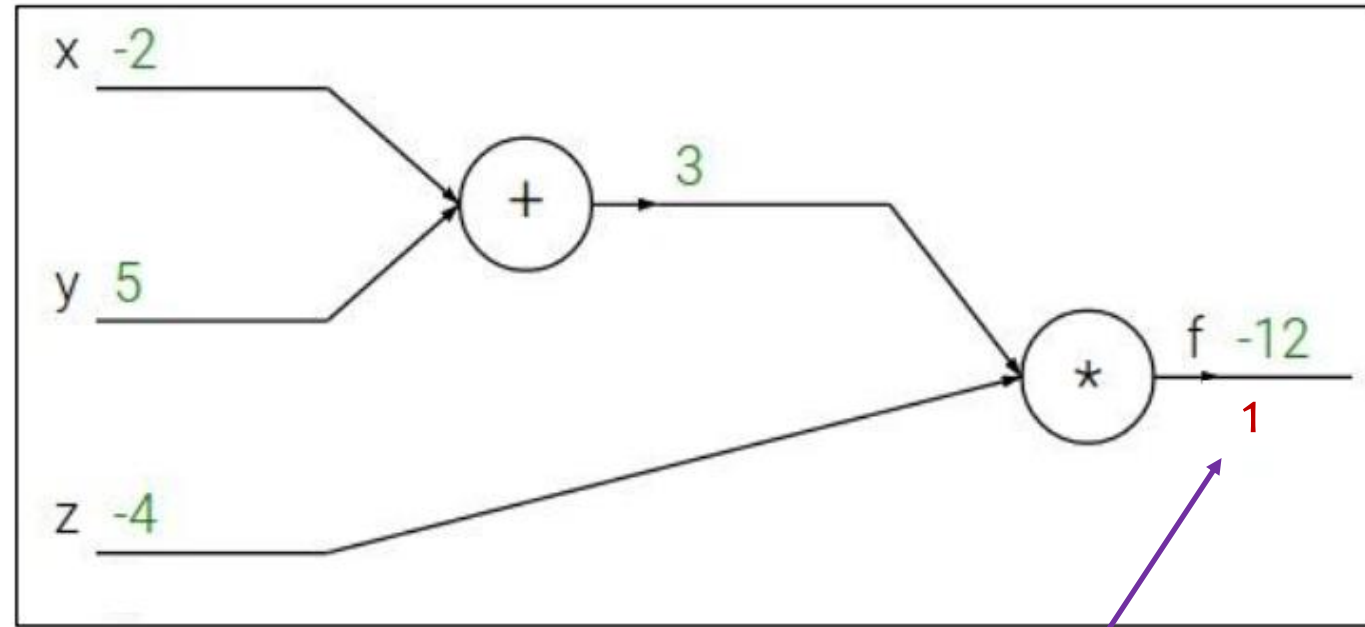
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

Backpropagation: simple example

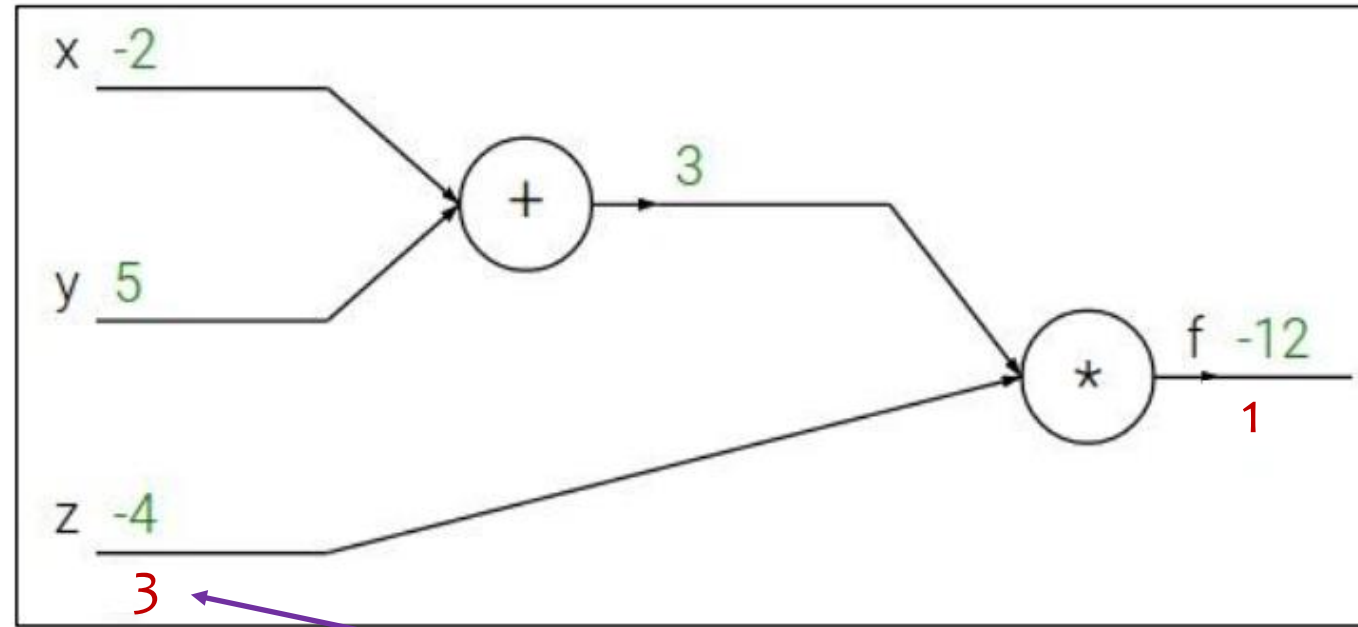
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backpropagation: simple example

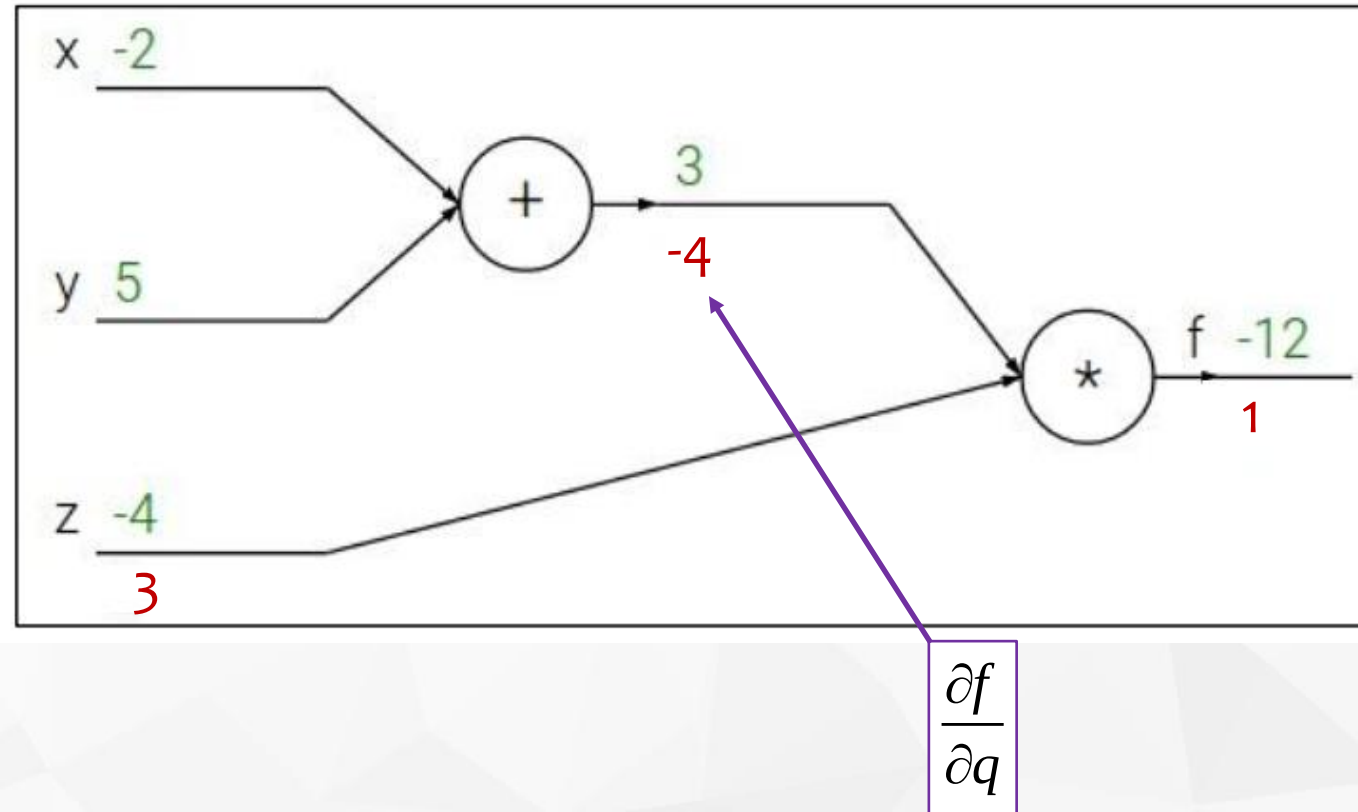
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: simple example

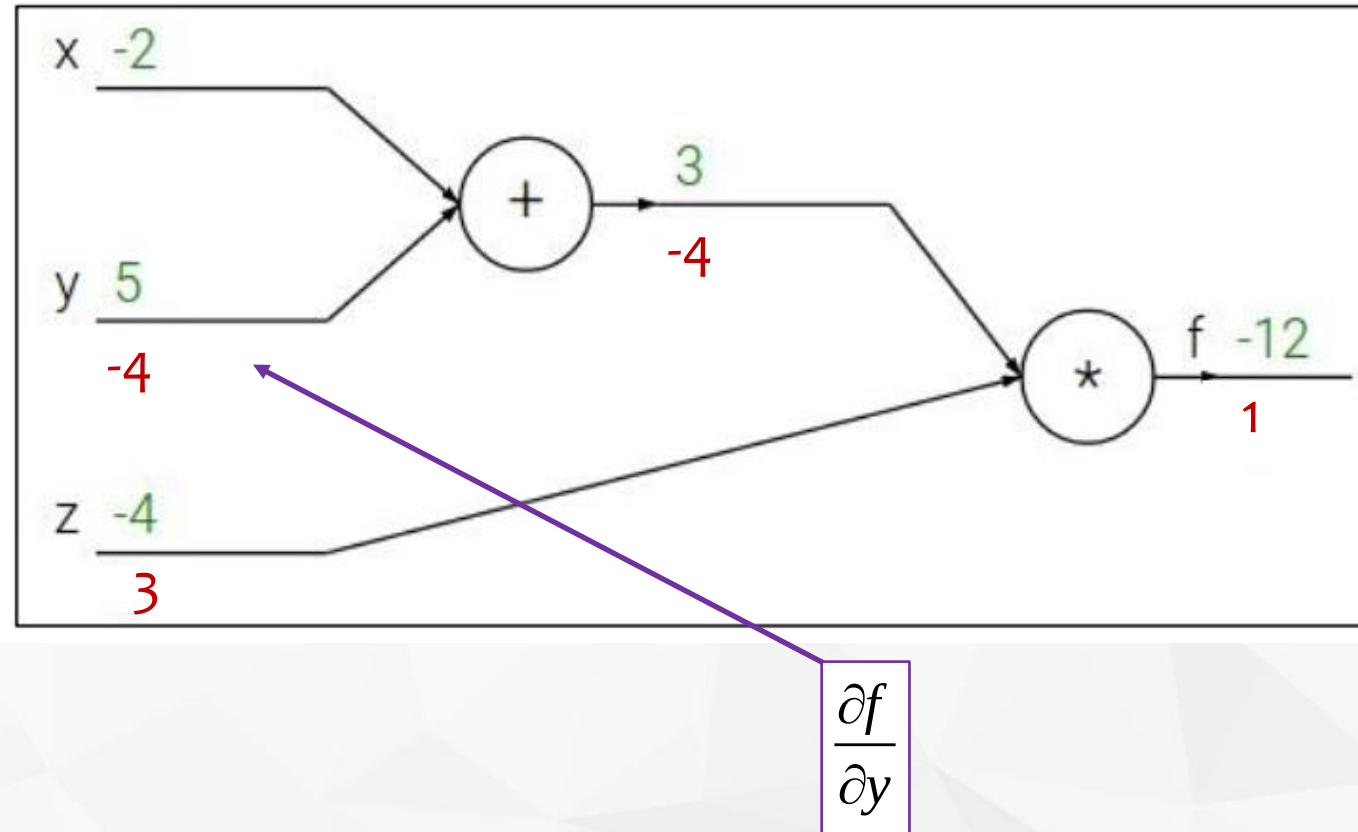
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Backpropagation: simple example

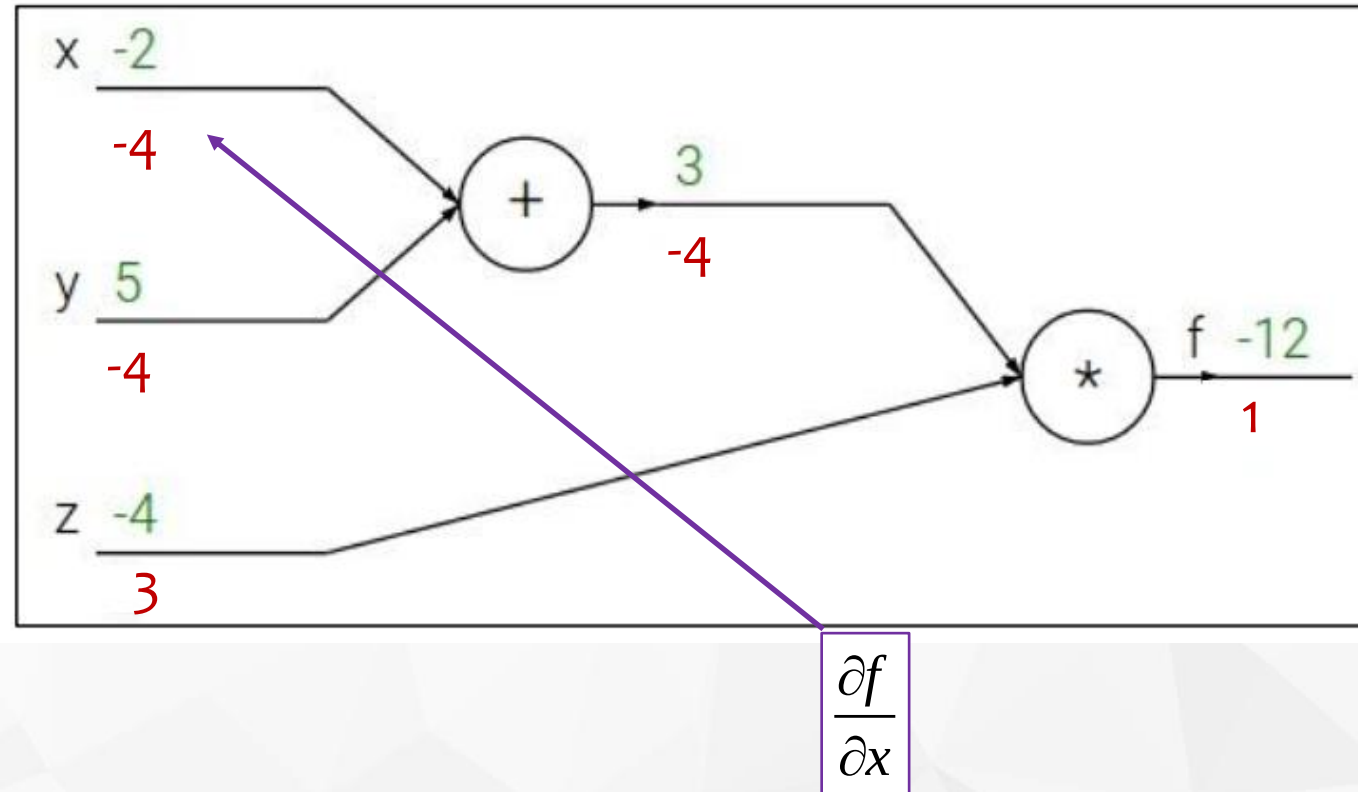
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

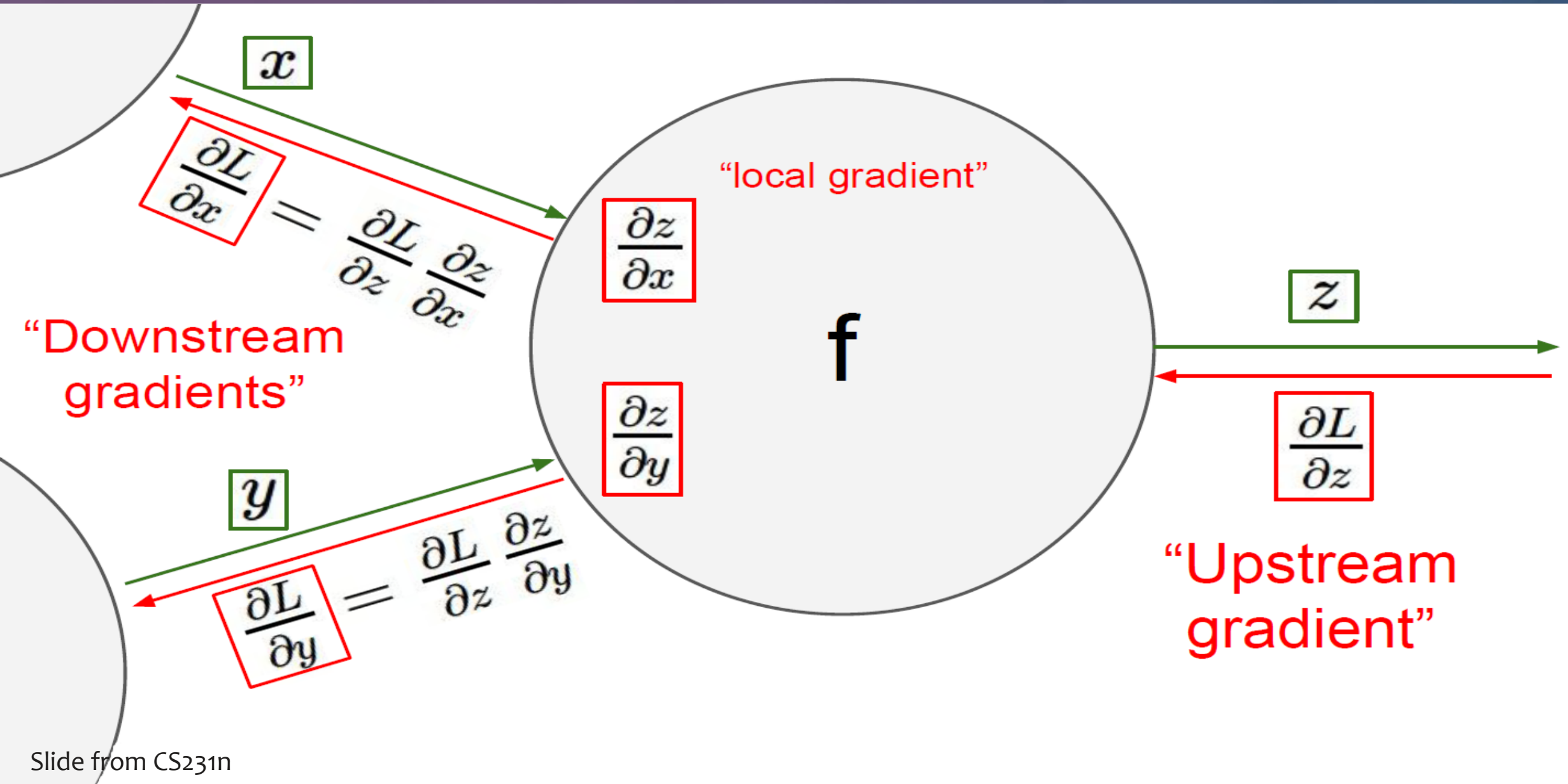
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

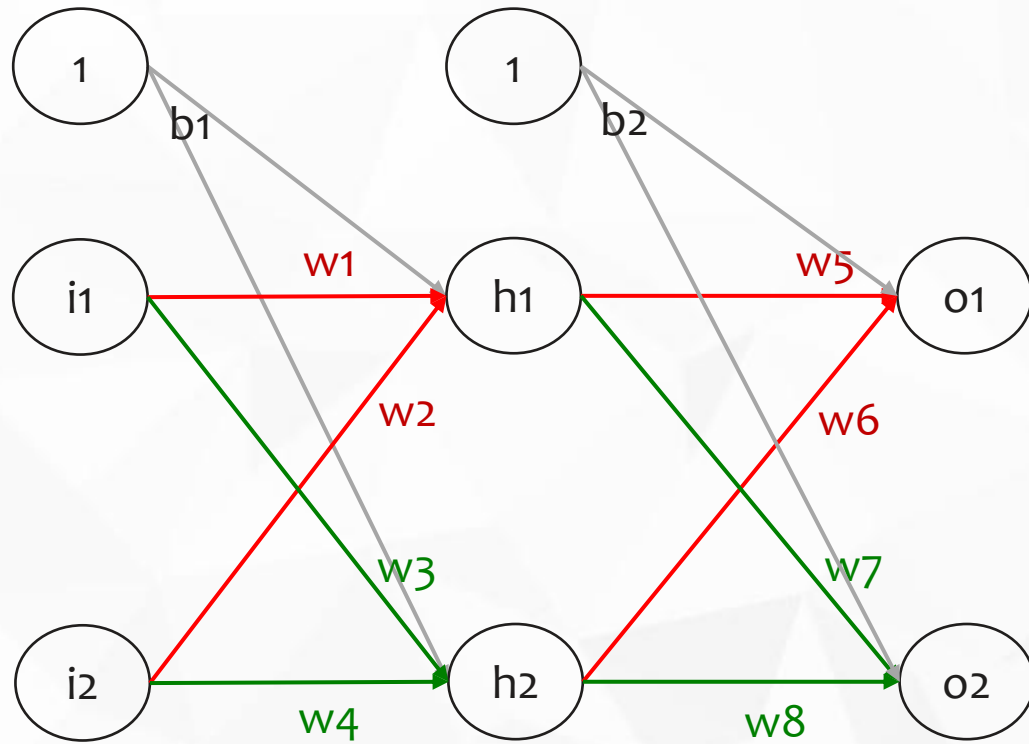


Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Backpropagation





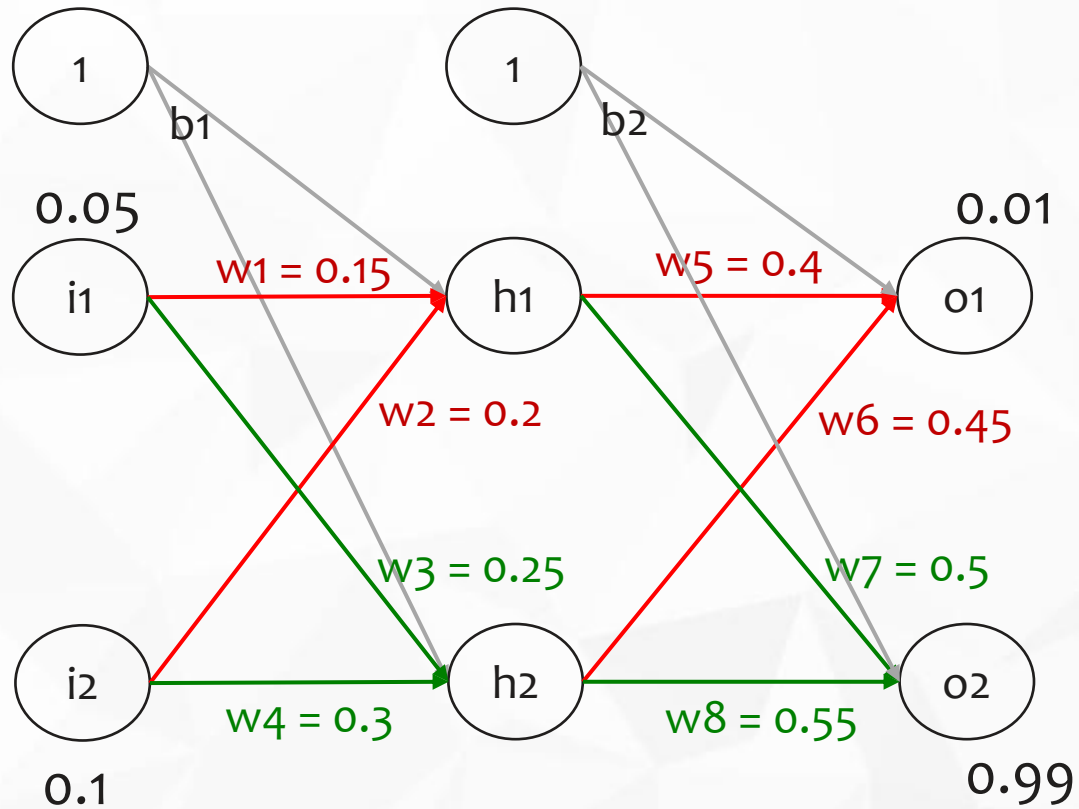
Simple example on backpropagation

Given inputs 0.05 and 0.10, we want the neural network to output 0.01 and 0.99.

→ How to find the weights and biases?

→ Backpropagation

Initial weights, the biases, and training inputs/outputs



Feedforward

$$\text{in}_{h1} = w_1 \times i_1 + w_2 \times i_2 + b_1 = 0.3775$$
$$\text{out}_{h1} = \text{sigmoid}(\text{in}_{h1}) = 0.593$$

$$\text{in}_{h2} = w_3 \times i_1 + w_4 \times i_2 + b_1$$
$$\text{out}_{h2} = \text{sigmoid}(\text{in}_{h2}) = 0.597$$

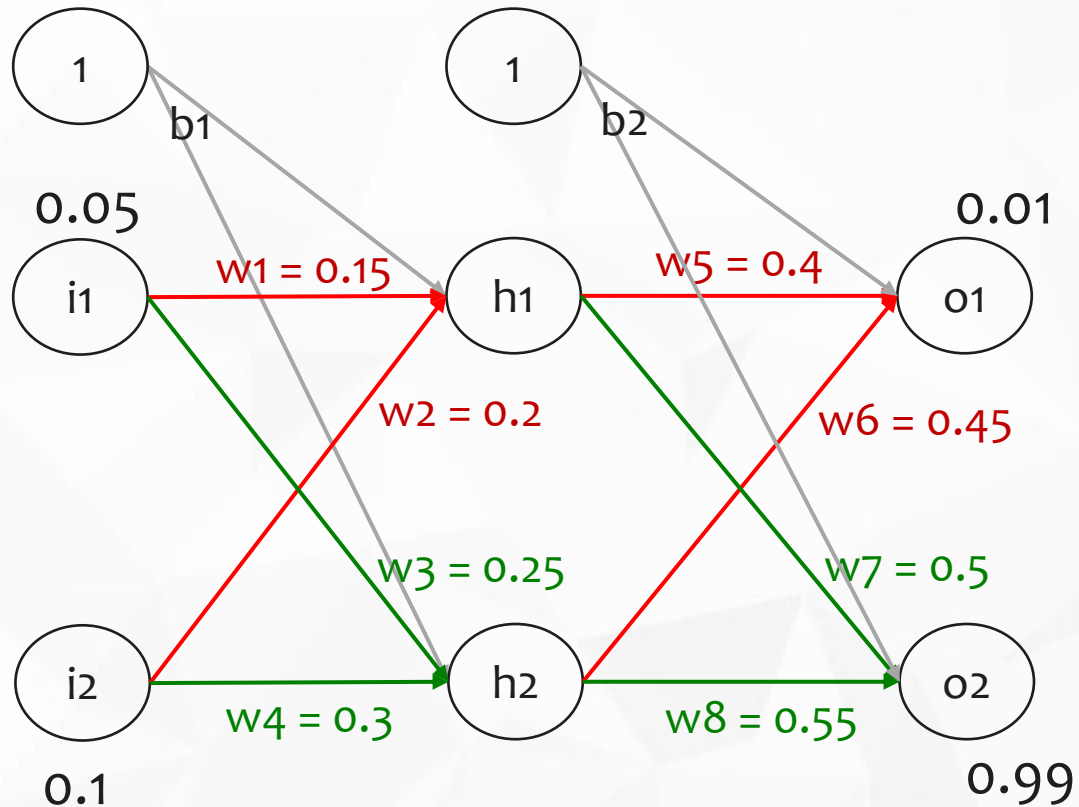
$$\text{in}_{o1} = w_5 \times \text{out}_{h1} + w_6 \times \text{out}_{h2} + b_2$$
$$\text{out}_{o1} = \text{sigmoid}(\text{in}_{o1}) = 0.751$$

$$\text{in}_{o2} = w_7 \times \text{out}_{h1} + w_8 \times \text{out}_{h2} + b_2$$
$$\text{out}_{o2} = \text{sigmoid}(\text{in}_{o2}) = 0.773$$

→ Total error:

$$\text{Error} = \frac{1}{2} \sum (\text{target} - \text{output})^2 = 0.3$$

Initial weights, the biases, and training inputs/outputs



Backpropagation

$$w_5^+ = w_5 - \alpha * \frac{\partial E_{\text{total}}}{\partial w_5}$$

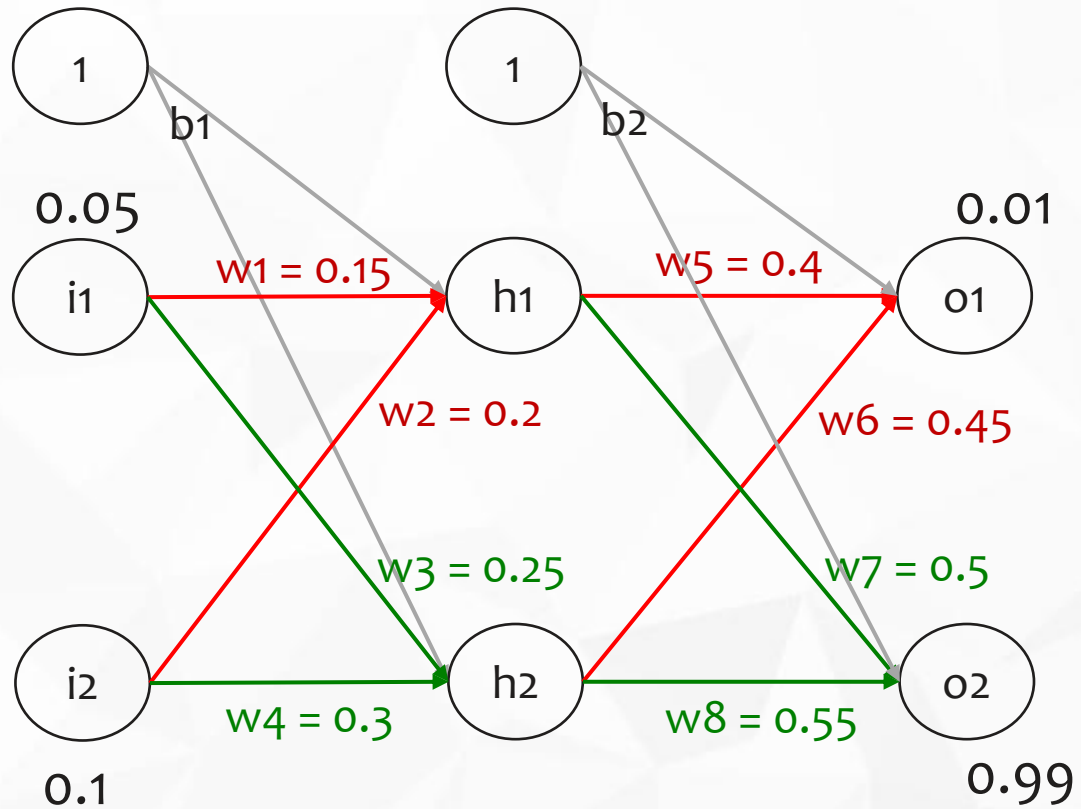
$$\frac{\partial E_{\text{total}}}{\partial w_5} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} \times \frac{\partial \text{out}_{o1}}{\partial \text{in}_{o1}} \times \frac{\partial \text{in}_{o1}}{\partial w_5}$$

$$\delta_{o1} = -(target_{o1} - out_{o1}) \times out_{o1} \times (1 - out_{o1})$$

$$\frac{\partial E_{\text{total}}}{\partial w_5} = \delta_{o1} \times out_{h1}$$

$$w_5^+ = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

Initial weights, the biases, and training inputs/outputs



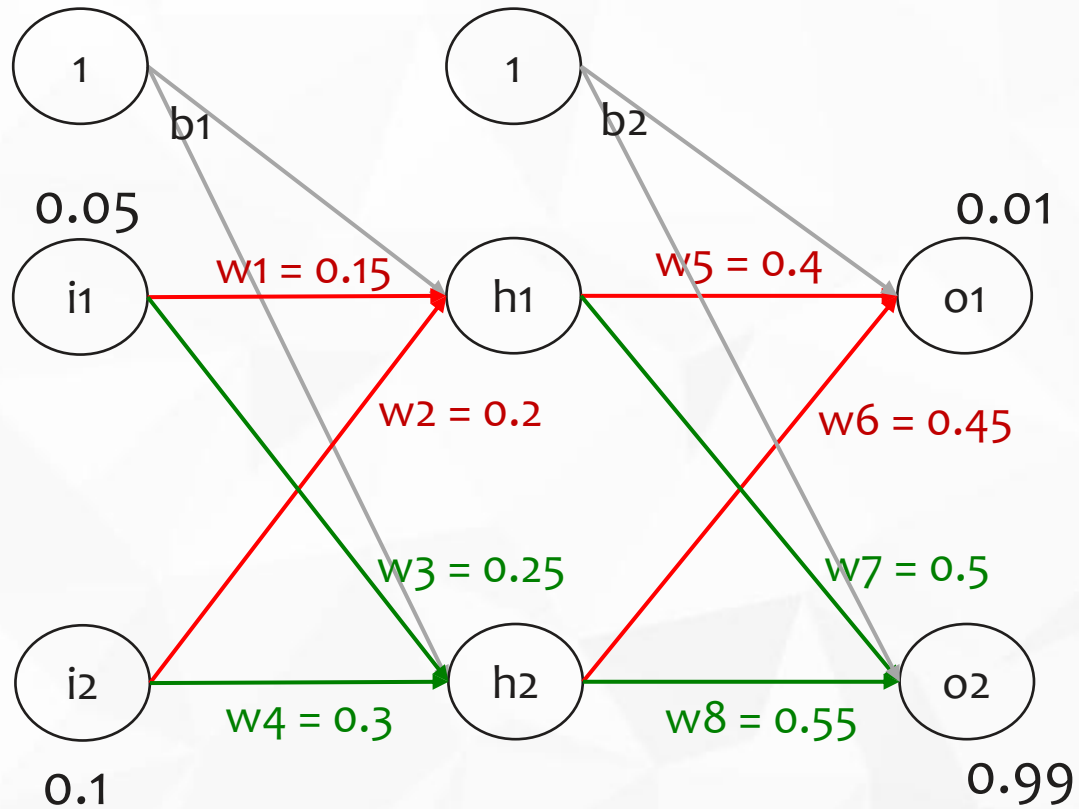
Backpropagation

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$

Initial weights, the biases, and training inputs/outputs



Backpropagation

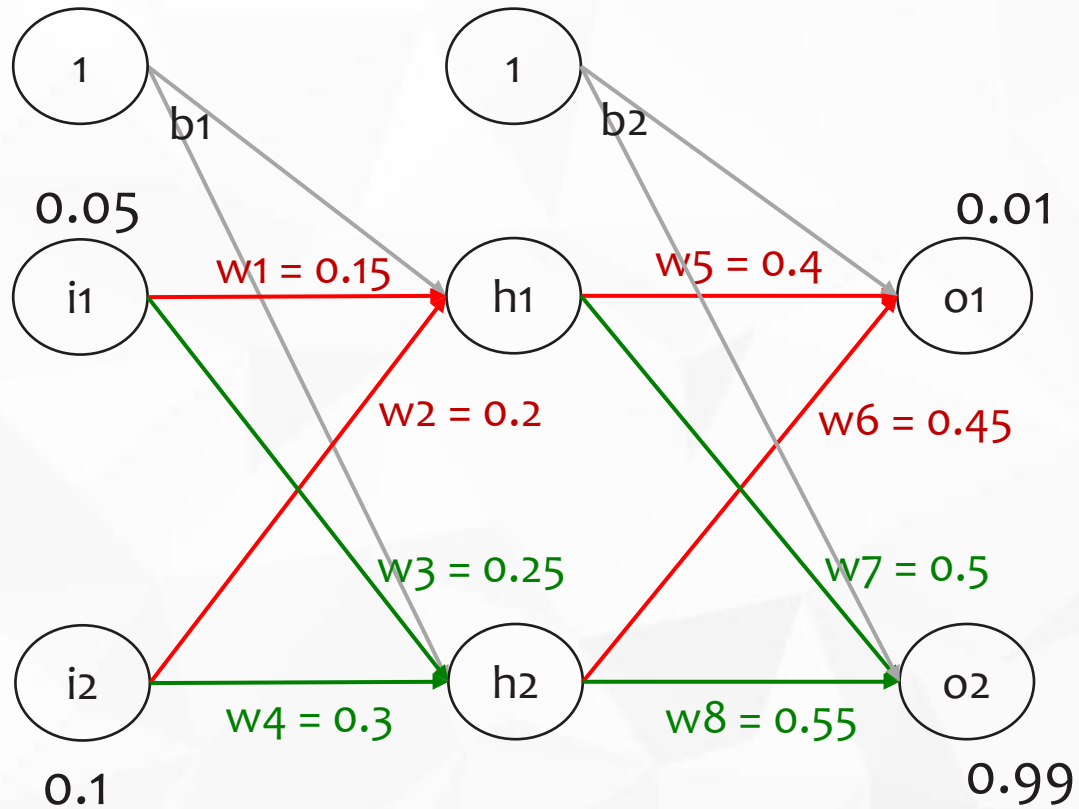
$$w_1^+ = w_1 - \alpha * \frac{\partial E_{\text{total}}}{\partial w_1}$$

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} * \frac{\partial \text{out}_{h1}}{\partial \text{in}_{h1}} * \frac{\partial \text{in}_{h1}}{\partial w_1}$$

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \left(\sum_o \delta_o \times w_{ho} \right) \times \text{out}_{h1} \times (1 - \text{out}_{h1}) * i1$$

$$\begin{aligned} w_1^+ &= 0.15 - 0.5 * 0.00043857 \\ &= 0.14978071 \end{aligned}$$

Initial weights, the biases, and training inputs/outputs



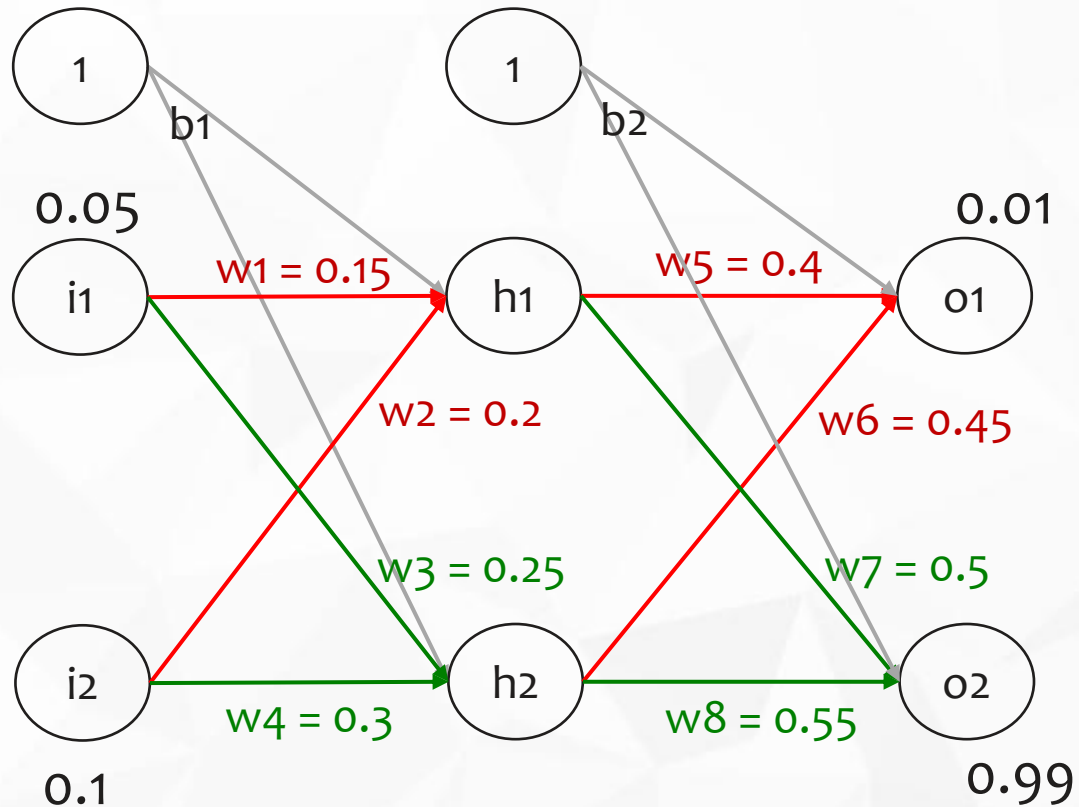
Backpropagation

$$w_2^+ = 0.19956143$$

$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$

Initial weights, the biases, and training inputs/outputs



```
class mlp():
```

```
# define the network with the given weights and biases
```

```
# input size = 2, output size = 2
```

```
# number of hidden layers = 1, number of neurons in the hidden layer = 2
```

```
# activation function in the hidden layer = sigmoid function
```

```
# activation function in the output layer = sigmoid function
```

```
def __init__(self):
```

```
# initialize the network weights and biases
```

```
...
```

```
self.weights = ...
```

```
self.biases = ...
```

```
# sigmoid function
```

```
def sigmoid(self, z):
```

```
...
```

```
# feedforward
```

```
def feedforward(self, input):
```

```
...
```

```
# backpropagation
```

```
def backpropagation(self, input, expected_output, learning_rate = 0.5):
```

```
# forward pass
```

```
...
```

```
# compute the total error
```

```
...
```

```
# backward pass
```

```
# output neuron delta (dE/dw5, dE/dw6, dE/dw7, dE/dw8)
```

```
...
```

```
# hidden neuron delta (dE/dw1, dE/dw2, dE/dw3, dE/dw4)
```

```
...
```

```
# update weights
```

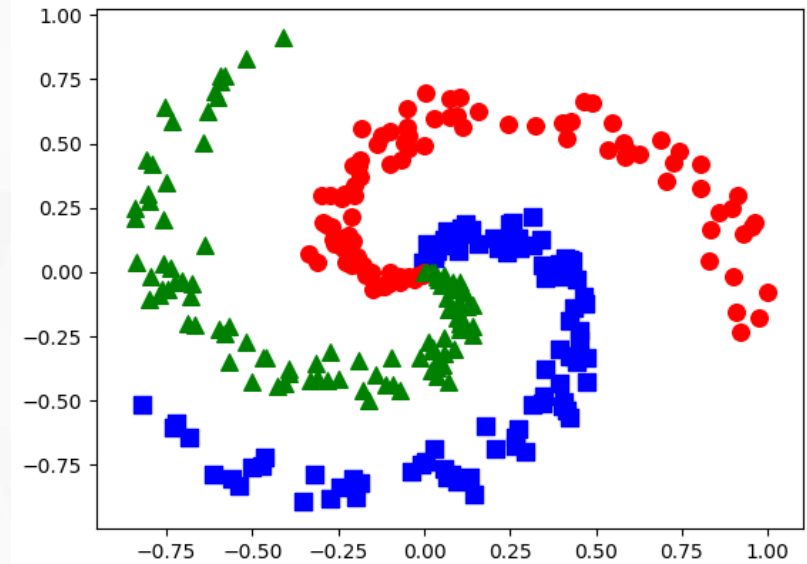
```
...
```

Lab: Neural Networks - Backpropagation

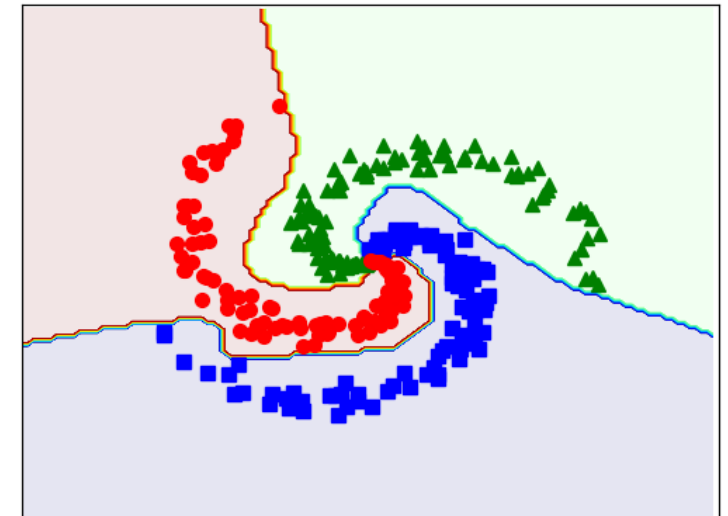
- In this lab, we will build a neural network from scratch and code how to find the optimal weights and biases using backpropagation.
- Please note that all deep learning libraries have the entire training and prediction processes implemented, and so in practice you wouldn't really need to build a neural network from scratch.
- Hopefully completing this lab will help you understand neural networks and how they work even better.

Lab: Neural Networks - Backpropagation

- Dataset: Google Classroom >> Classwork >> [Bài 1] Thực hành >> backpropagation.txt
- Compute the accuracy in the following cases:
 - Number of hidden units = 5
 - Number of hidden units = 10
 - Number of hidden units = 20
 - Number of hidden units = 50
 - Number of hidden units = 100



#hidden units = 200, accuracy = 99.33 %



```
def softmax(V):  
    e_V = np.exp(V - np.max(V, axis = 0, keepdims = True))  
    Z = e_V / e_V.sum(axis = 0)  
    return Z  
  
## One-hot coding  
from scipy import sparse  
def convert_labels(y, C = 3):  
    Y = sparse.coo_matrix((np.ones_like(y),  
        (y, np.arange(len(y)))), shape = (C, len(y))).toarray()  
    return Y  
  
# cost or loss function  
def cost(Y, Yhat):  
    return -np.sum(Y*np.log(Yhat))/Y.shape[1]
```

```

# define and train the network
# number of the hidden layers = 1
class mlp():
    def __init__(self, input_size, hidden_layer_size, output_size):
        self.input_size = ...
        self.hidden_layer_size = ...
        self.output_size = ...
        # initialize weights and biases randomly
        W1 = 0.01*np.random.randn(input_size, hidden_layer_size)
        b1 = np.zeros((hidden_layer_size, 1))
        W2 = ...
        b2 = ...
        self.weights = ...
        self.biases = ...

    def backpropagation(self, input, expected_output, learning_rate, n_iter):
        for i in range(n_iter):
            # forward pass -- feedforward
            Z1 = np.dot(self.weights[0].T, input) + self.biases[0]
            A1 = np.maximum(Z1, 0)
            Z2 = np.dot(self.weights[1].T, A1) + self.biases[1]
            Yhat = softmax(Z2)
            # backward pass
            # compute gradient
            N = input.shape[1]
            E2 = (Yhat - expected_output)/N
            dW2 = np.dot(A1, E2.T)
            db2 = np.sum(E2, axis = 1, keepdims = True)
            E1 = np.dot(self.weights[1], E2)
            E1[Z1 <= 0] = 0 # gradient of ReLU
            dW1 = np.dot(input, E1.T)
            db1 = np.sum(E1, axis = 1, keepdims = True)
            # update weights and biases
            ...

    def train(self, input, expected_output, learning_rate = 1, n_iter = 10000):
        # one hot encoding
        Y = convert_labels(expected_output, self.output_size)
        # training
        self.backpropagation(input, Y, learning_rate, n_iter)

```

Hands-on practices with sklearn package

scikit-learn
Machine Learning in Python

[Getting Started](#)

[Release Highlights for 0.24](#)

[GitHub](#)

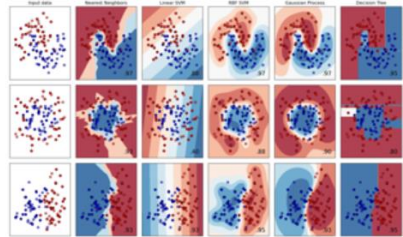
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: SVM, nearest neighbors, random forest, and more...



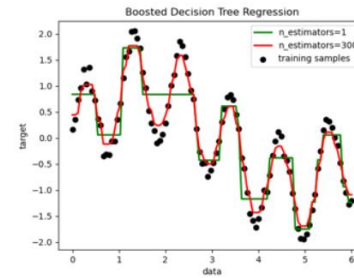
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, nearest neighbors, random forest, and more...



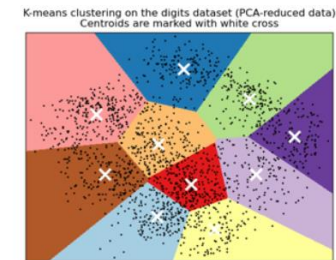
Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, and more...



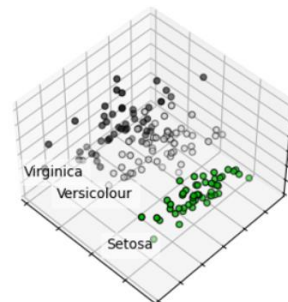
Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: k-Means, feature selection, non-negative matrix factorization, and more...



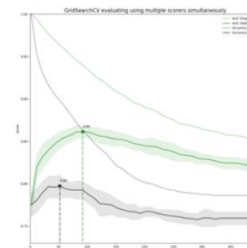
Examples

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning

Algorithms: grid search, cross validation, metrics, and more...



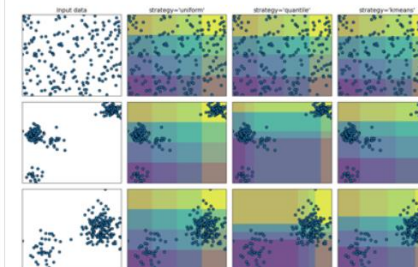
Examples

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

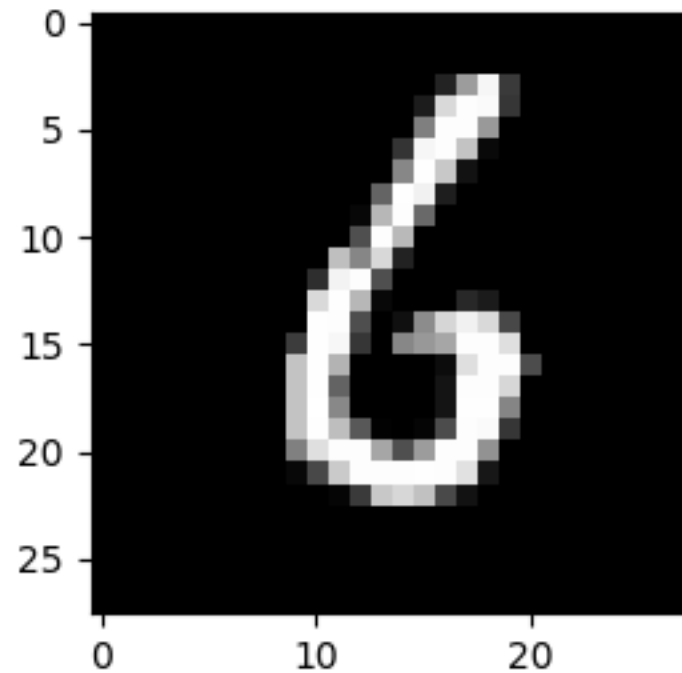
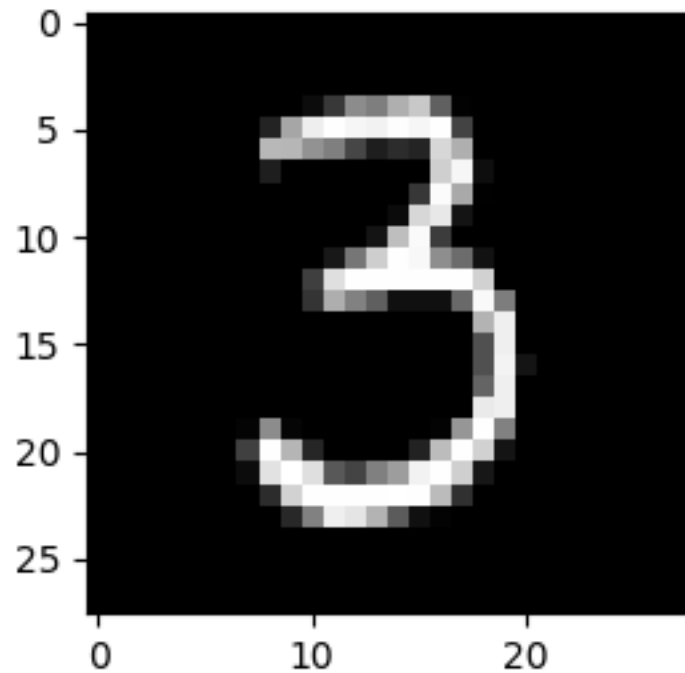
Algorithms: preprocessing, feature extraction, and more...



Examples

Neural network for digit recognition

Training image: 3 Testing image: 6



Neural network for digit recognition

1. Load digit dataset provided by sklearn
2. Split the dataset: 80% for training, 20% for testing
3. Normalize the feature values
4. Design and train the network
5. Evaluate the trained model

```
mlp = MLPClassifier(hidden_layer_sizes=(2),  
                    activation = 'relu',  
                    learning_rate_init = 0.001,  
                    max_iter = 2000,  
                    random_state=2,  
                    n_iter_no_change = 2000)
```

```
mlp.fit(X_train, y_train)  
acc = mlp.score(X_test, y_test)
```

Diabetes Detection

Dataset: Google Classroom >> Classwork >> [Bài 1] Thực hành >> diabetes.txt

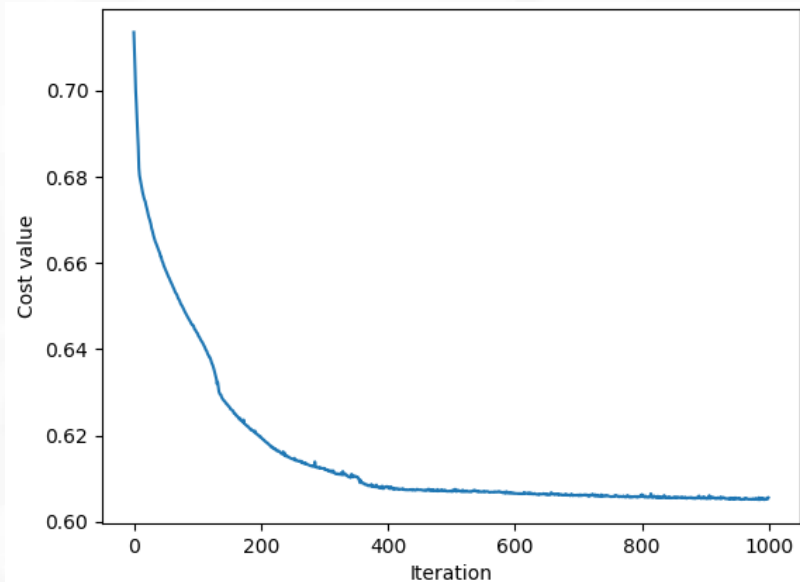
The dataset contains 768 observations and 8 variables, as described below:

1. pregnancies - Number of times pregnant.
2. glucose - Plasma glucose concentration.
3. diastolic - Diastolic blood pressure (mm Hg).
4. triceps - Skinfold thickness (mm).
5. insulin - Hour serum insulin (mu U/ml).
6. bmi – Basal metabolic rate (weight in kg/height in m).
7. dpf - Diabetes pedigree function.
8. age - Age in years.

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	ClassLabel
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1

1. Design and train a network that is suitable for detecting diabetes.
2. Explore the effect of the parameters in training the network.
3. Can feature normalization improve the performance of the network? (use StandardScaler in sklearn to normalize feature)

```
19 # train the mlp
20 mlp = MLPClassifier(hidden_layer_sizes=(2), activation='relu', learning_rate_init = 0.001
21                      , max_iter=1000, random_state=2, n_iter_no_change = 1000, verbose=False)
22 mlp.fit(X_train,y_train)
```



```
Weights = [array([[ 0.22825328,  0.10444592],
                  [ 0.81153388, -0.53293876],
                  [-0.07471738,  0.24236804],
                  [-0.19830177, -0.36181783],
                  [ 0.25740898,  0.6558339 ],
                  [ 0.52929212, -0.27144834],
                  [ 0.30588133,  0.02452766],
                  [-0.25641209, -1.54052567]]), array([[ 1.24737595],
                  [-2.30631131]])]
Biases = [array([ 1.03011901, -0.6314108 ]), array([-1.6745632])]
Size of output layer: 1
Activation function of output layer: logistic
```

Feature normalization

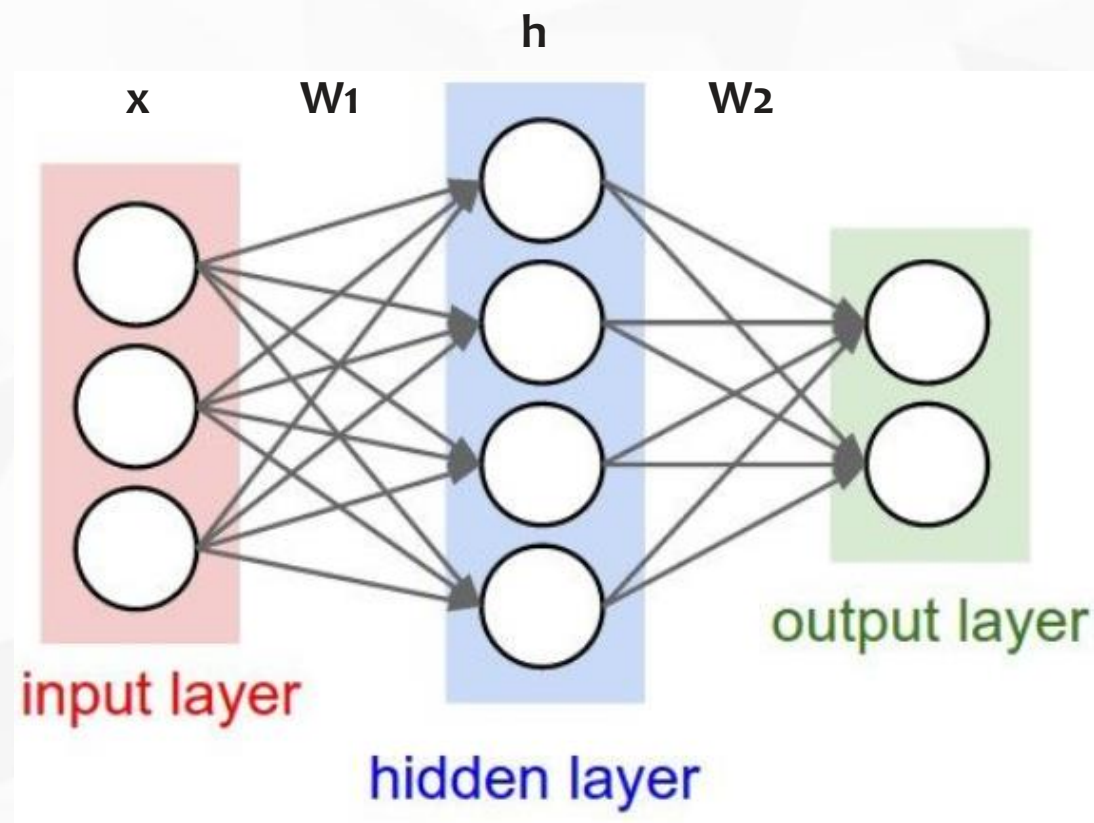
```
# normalize the feature values  
scaler = StandardScaler()  
scaler.fit(X_train)  
X_train = scaler.transform(X_train)  
X_test = scaler.transform(X_test)
```

More Discussion

Weight Initialization

Early neural networks didn't work quite as well as
because people were not careful enough with
weight initialization

Question #2: what happens when $W=0$ init is used?

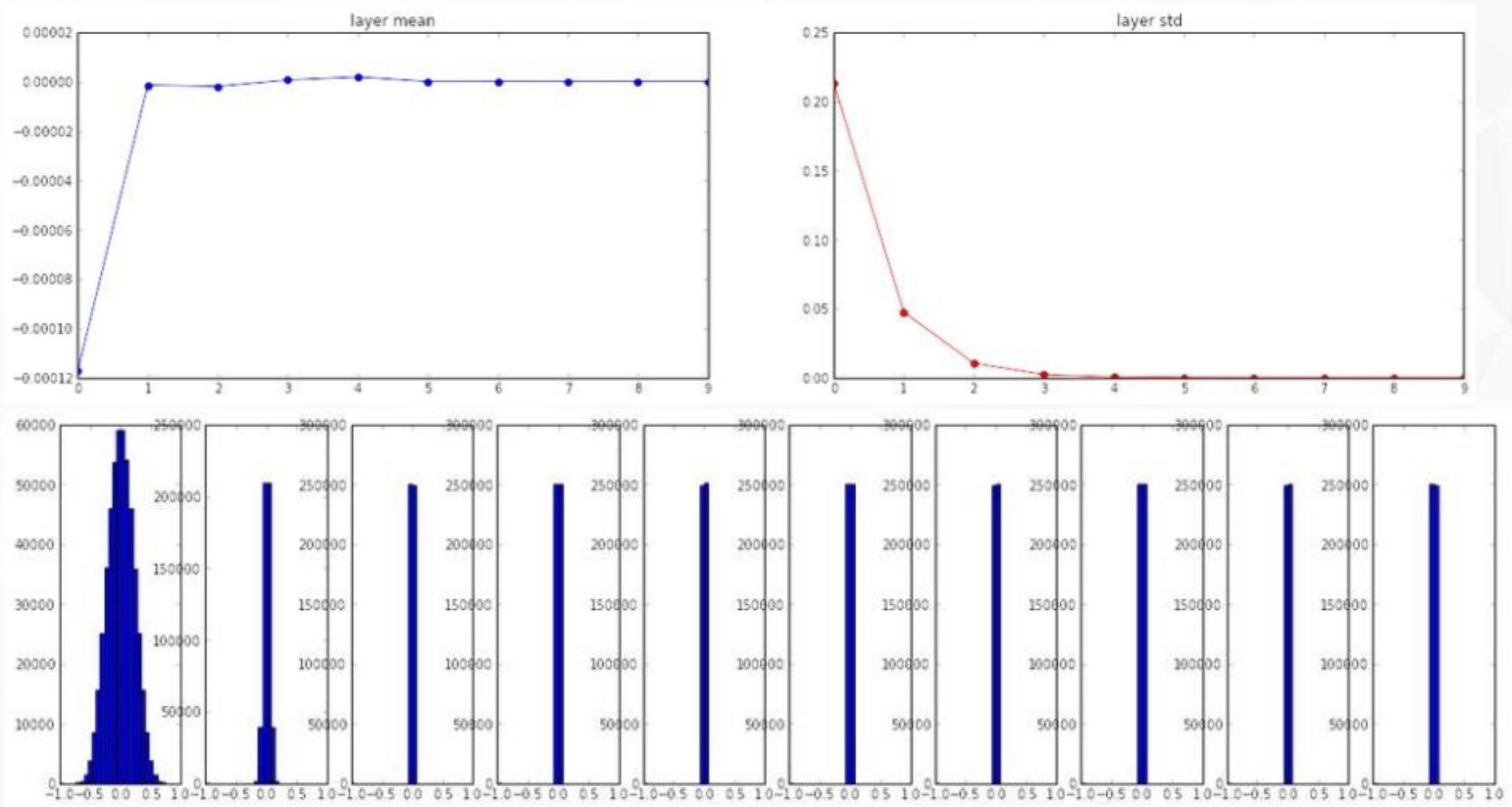


- Weight
- First idea: **Small random numbers**

(Gaussian with zero mean and 1e-2 standard deviation)

$$W = 0.01 * \text{np.random.randn}(D, H)$$

- Works ~okay for small networks, but can lead to **non-homogeneous** distributions of activations across the layers of a network

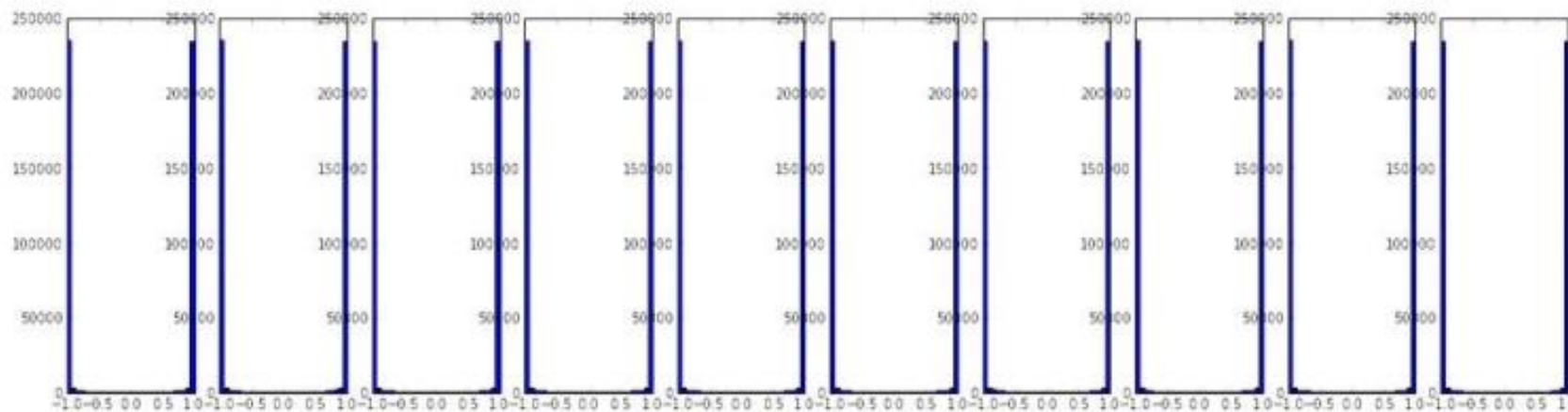
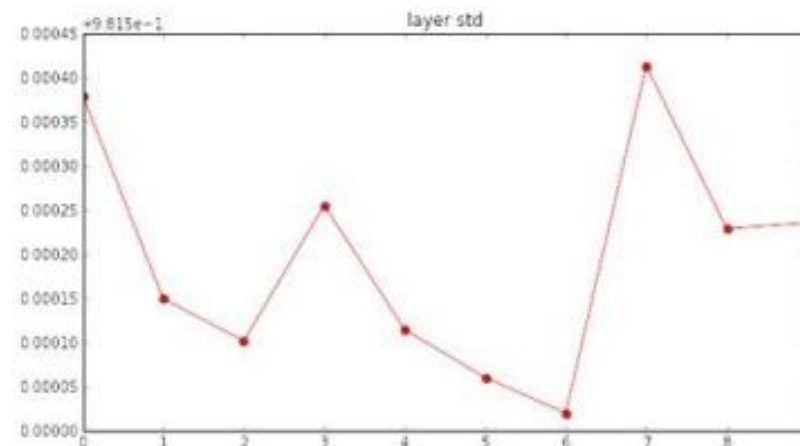
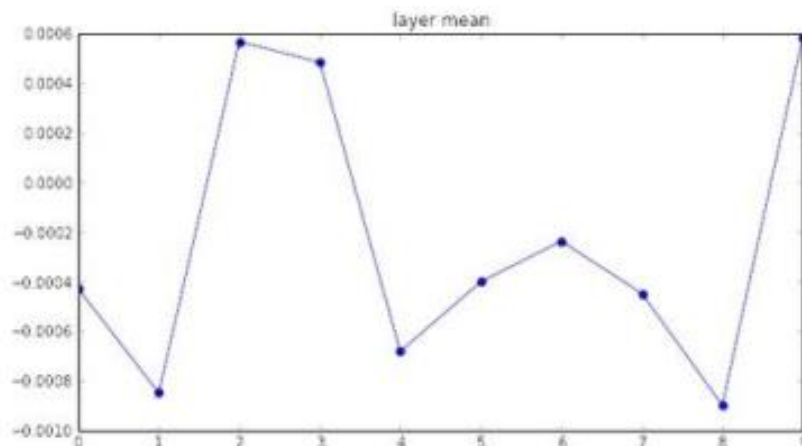


All activations become zero!

`W = np.random.randn(fan_in, fan_out) * 1.0 # layer initialization`

input layer had mean 0.001800 and std 1.001311
 hidden layer 1 had mean -0.000430 and std 0.981879
 hidden layer 2 had mean -0.000849 and std 0.981649
 hidden layer 3 had mean 0.000566 and std 0.981601
 hidden layer 4 had mean 0.000483 and std 0.981755
 hidden layer 5 had mean -0.000682 and std 0.981614
 hidden layer 6 had mean -0.000401 and std 0.981560
 hidden layer 7 had mean -0.000237 and std 0.981520
 hidden layer 8 had mean -0.000448 and std 0.981913
 hidden layer 9 had mean -0.000899 and std 0.981728
 hidden layer 10 had mean 0.000584 and std 0.981736

*1.0 instead of *0.01

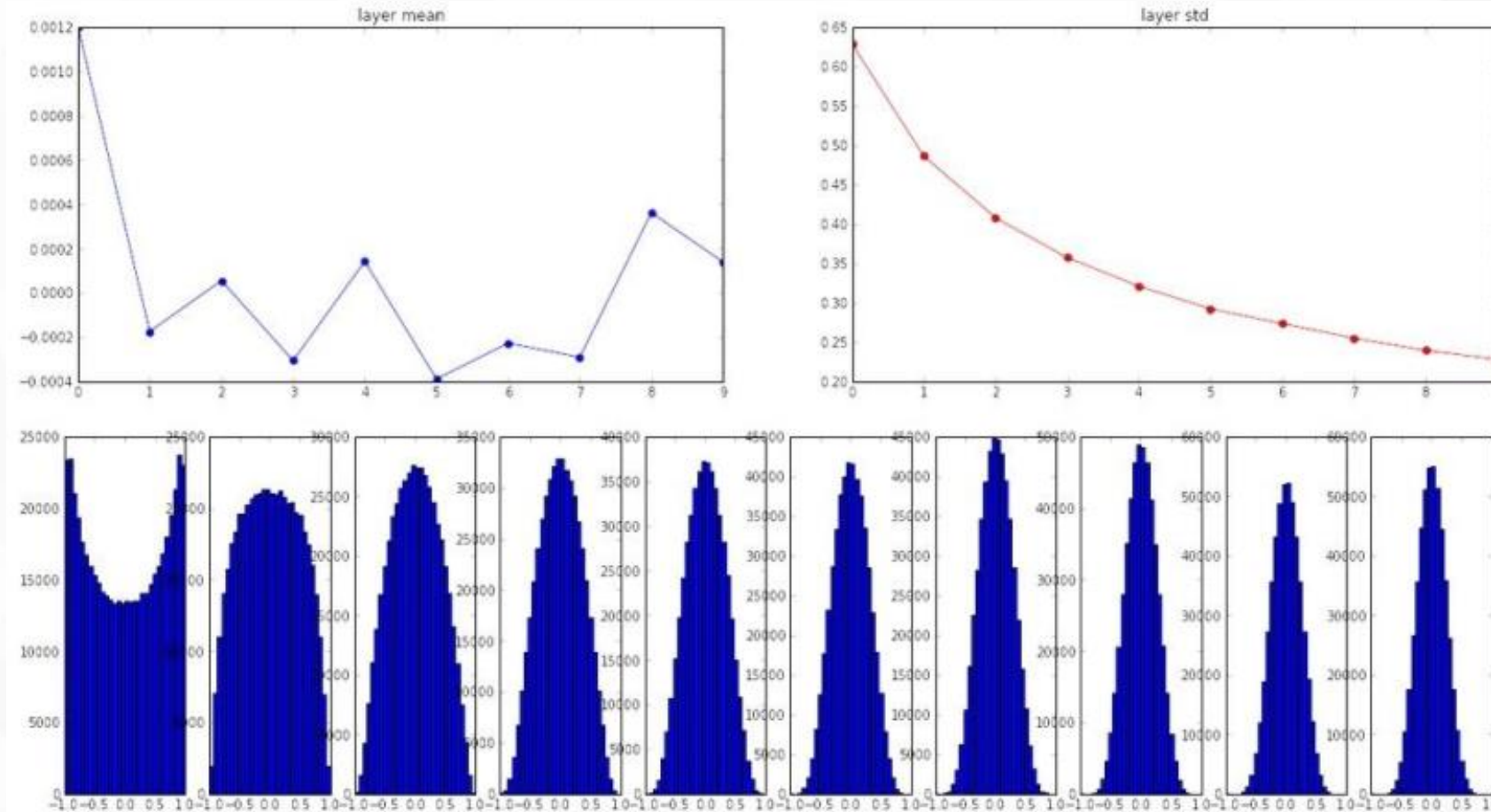


Almost all neurons **completely saturated**, either -1 and 1. Gradients will be all zero.

```
W = np.random.randn(fan_in, fan_out) / np.sqrt(fan_in) # layer initialization
```

“Xavier initialization”

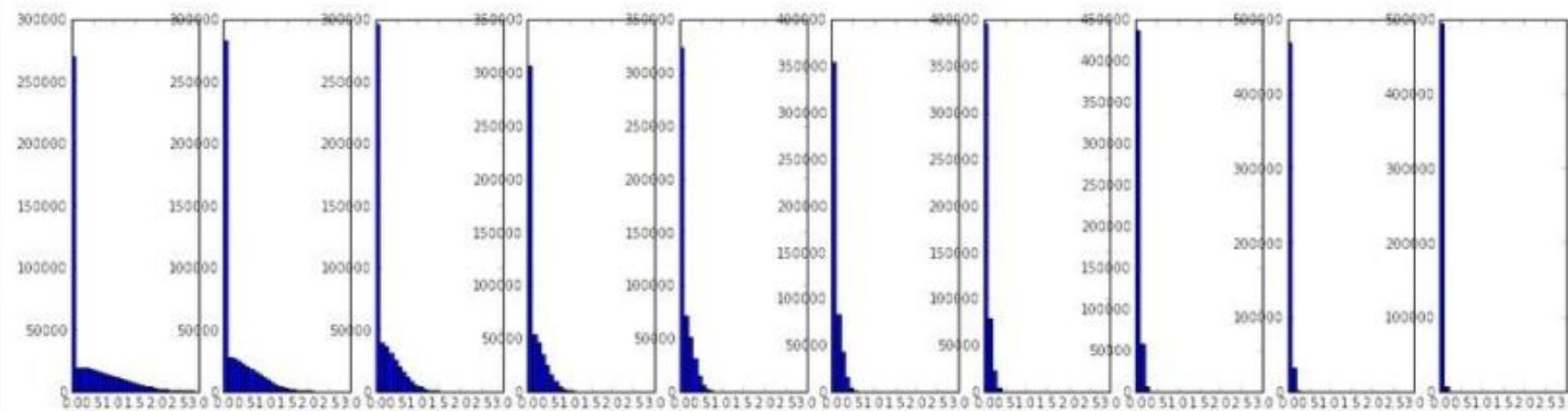
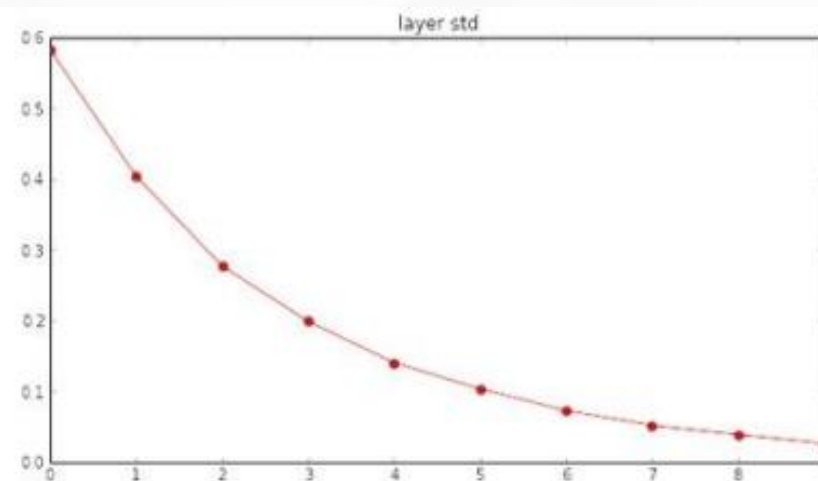
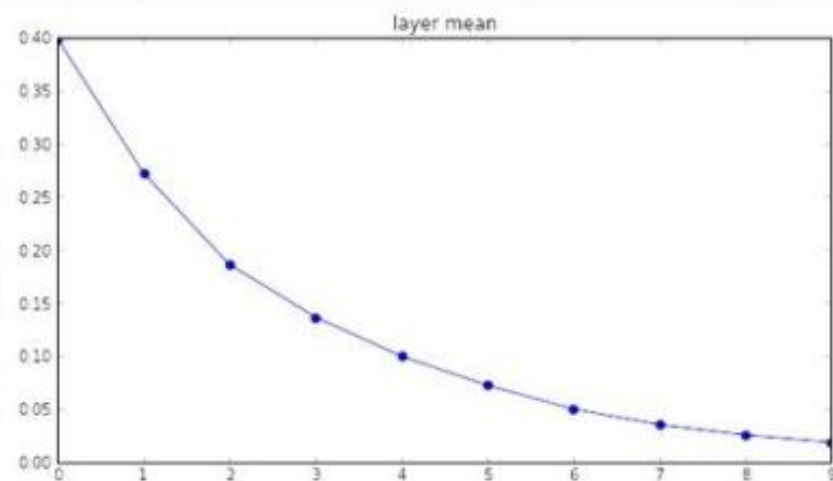
[Glorot et al., 2010]



Reasonable initialization.
(Mathematical derivation
assumes **linear**
activations)

```
W = np.random.randn(fan_in, fan_out) / np.sqrt(fan_in) # layer initialization
```

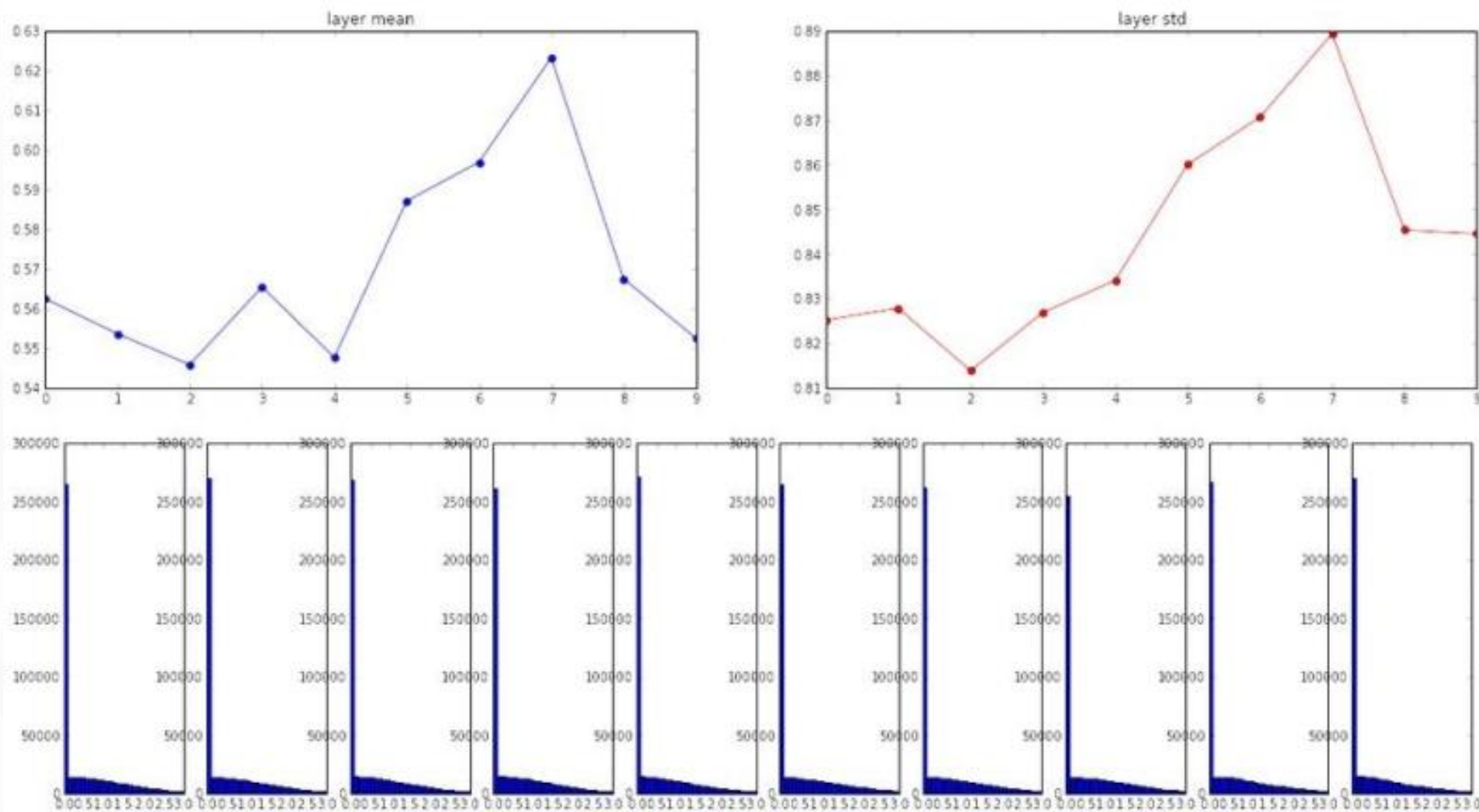
“Xavier initialization”



But when using
the **ReLU**
nonlinearity it
breaks


```
W = np.random.randn(fan_in, fan_out) / np.sqrt(fan_in/2) # layer initialization
```

He et al. 2015

(note additional/2)

We Proper initialization is an active area of research...

- **Understanding the difficulty of training deep feedforward neural networks** by Glorot and Bengio, 2010
- **Exact solutions to the nonlinear dynamics of learning in deep linear neural networks**, by Saxe et al, 2013
- **Random walk initialization for training very deep feedforward networks**, by Sussillo and Abbott, 2014
- **Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification** by He et al., 2015
- **Data-dependent initializations of Convolutional Neural Networks** by Krahenbuhl et al., 2015
- **All you need is good init**, Mishkin and Matas, 2015

Parameter Updates

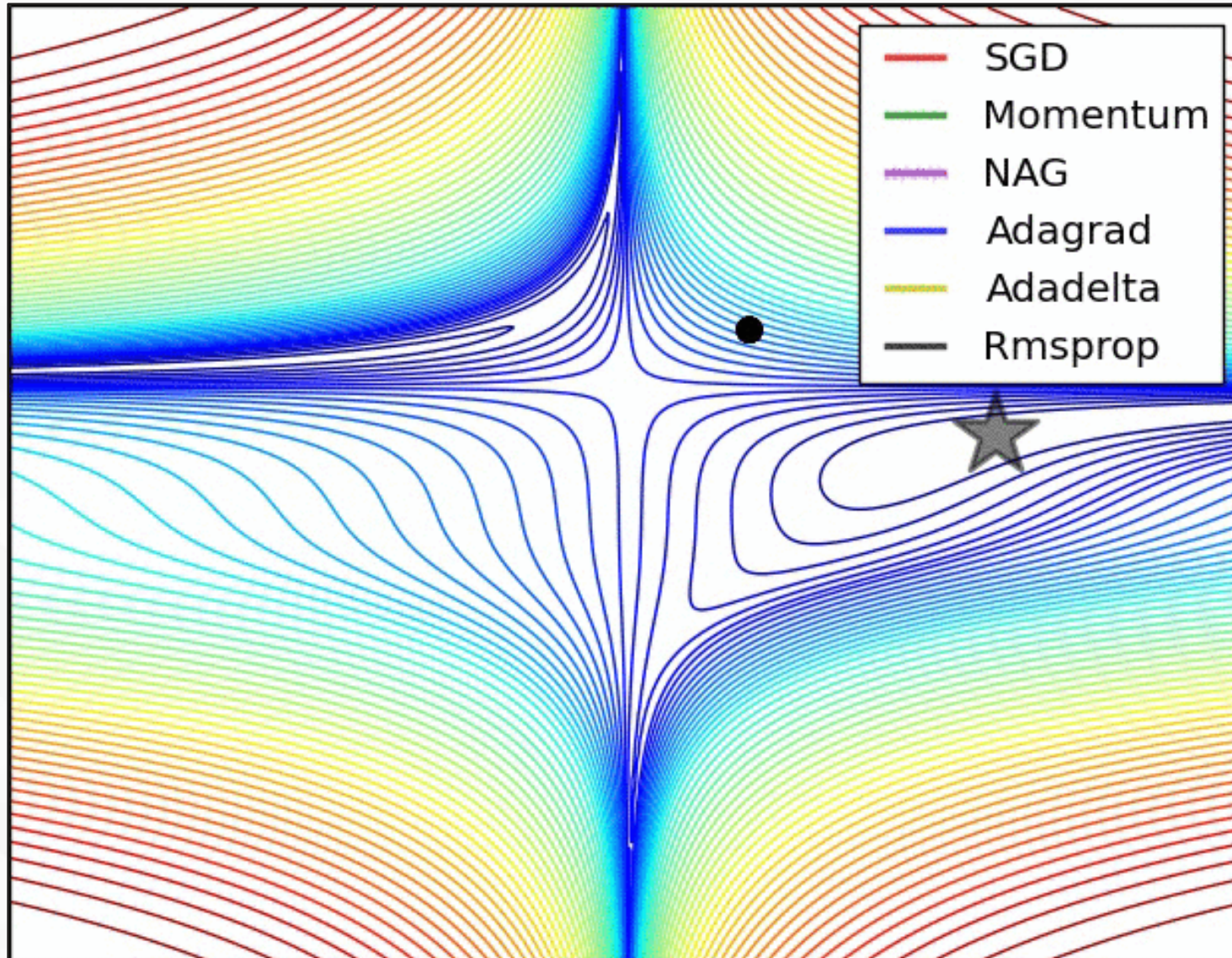
Parameter Updates

```
while True:  
    data_batch = dataset.sample_data_batch()  
    loss = network.forward(data_batch)  
    dx = network.backward()  
    x += - learning_rate * dx
```

Simple gradient descent update (SGD)

Now: complicate.

Parameter Updates

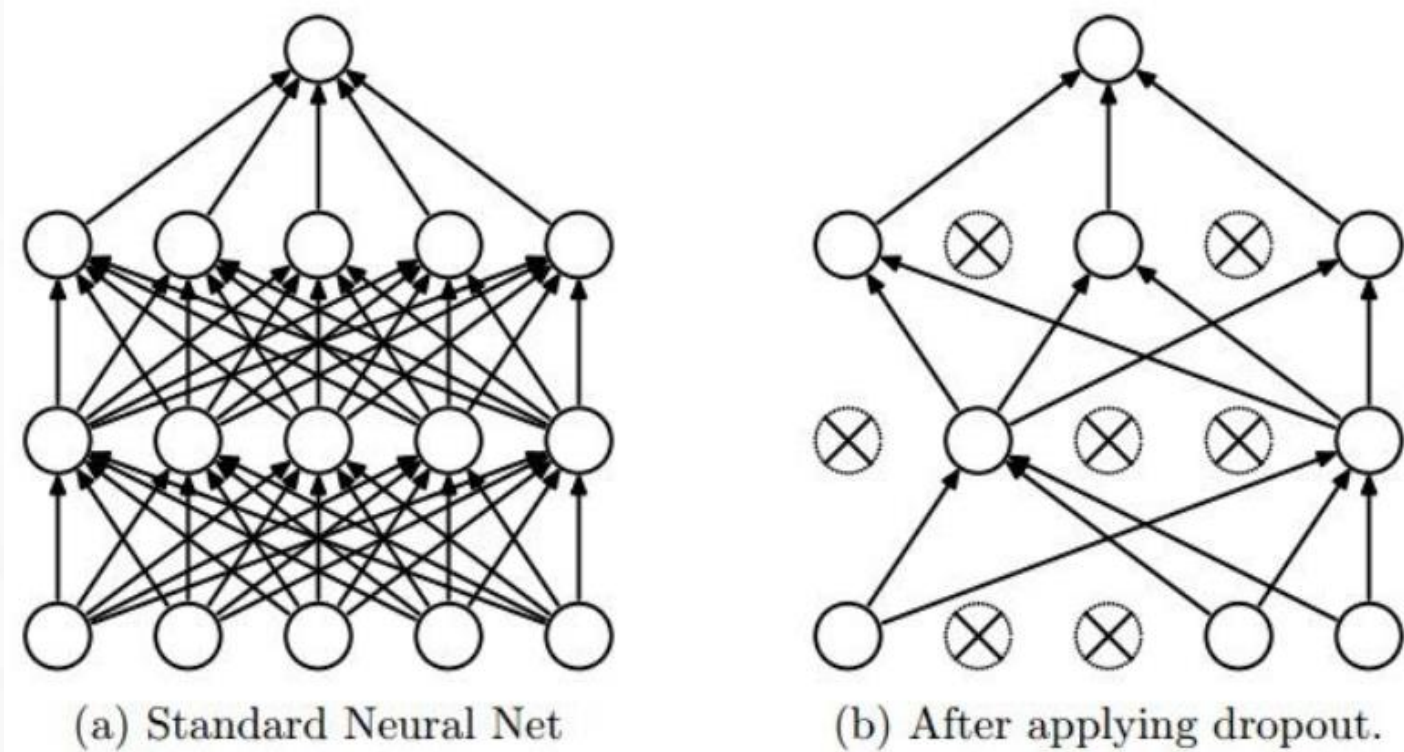


Adam is a good default choice in most cases

Regularization Dropout

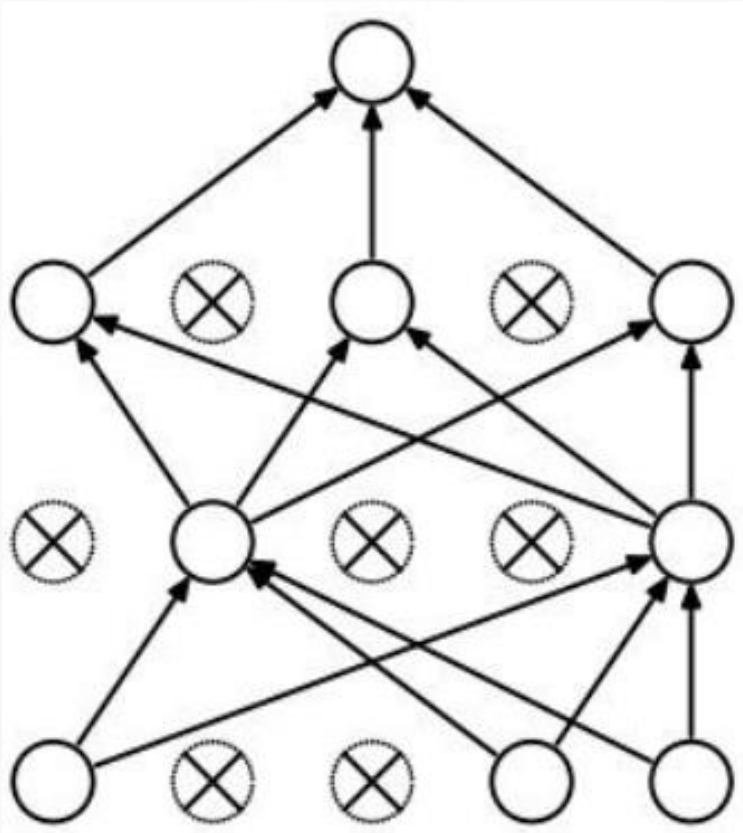
Regularization: Dropout

“Randomly set some neurons to zero in the forward pass”



Regularization: Dropout

Question: how could this possible be a good idea?



- Forces the network to have a **redundant representation**

